

Trees and Tree Algorithms

1. The Tree Abstract Data Type
2. Tree Traversals
3. Priority Queue and Binary Heap
4. Binary Search Trees
5. AVL tree

8.2. Examples of Trees

Trees are used in many areas of computer science, including operating systems, graphics, database systems, and computer networking. A tree data structure has a root, branches, and leaves. The difference between a tree in nature and a tree in computer science is that a tree data structure has its root at the top and its leaves on the bottom!

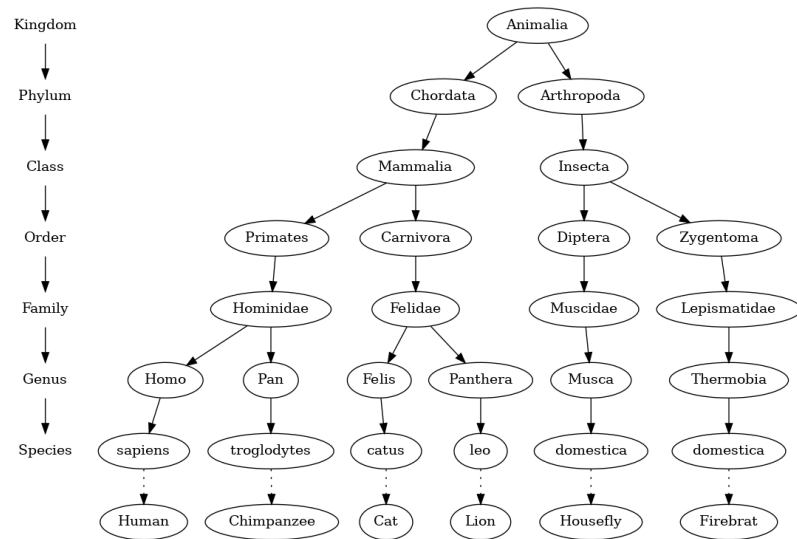
Trees are used in many areas of computer science, including operating systems, graphics, database systems, and computer networking. A tree data structure has a root, branches, and leaves. The difference between a tree in nature and a tree in computer science is that a tree data structure has its root at the top and its leaves on the bottom!

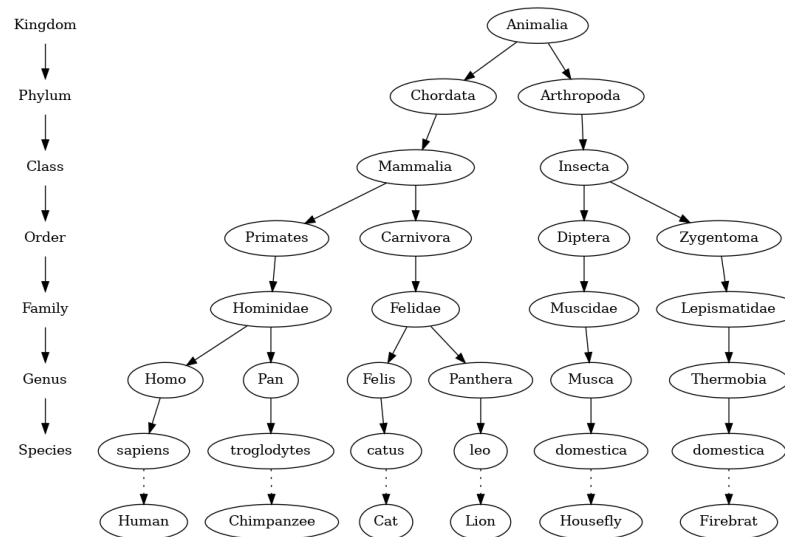
Our first example of a tree is a classification tree from biology which shows an example of the biological classification of some animals.

Trees are used in many areas of computer science, including operating systems, graphics, database systems, and computer networking. A tree data structure has a root, branches, and leaves. The difference between a tree in nature and a tree in computer science is that a tree data structure has its root at the top and its leaves on the bottom!

Our first example of a tree is a classification tree from biology which shows an example of the biological classification of some animals.

This example demonstrates that trees are **hierarchical**. By hierarchical, we mean that trees are structured in layers with the more general things near the top and the more specific things near the bottom. The top of the hierarchy is the kingdom, the next layer of the tree (the "children" of the layer above) is the phylum, then the class, and so on.



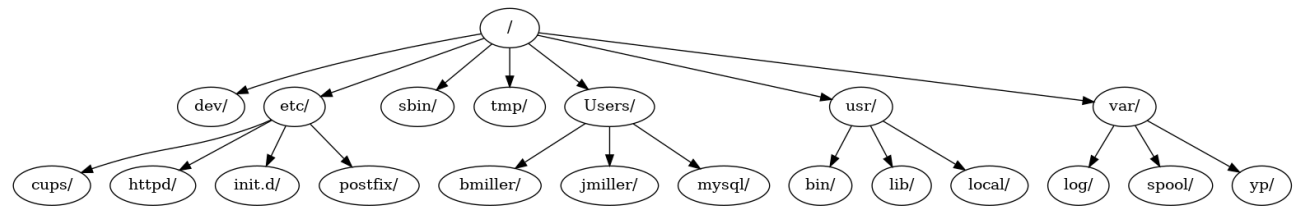


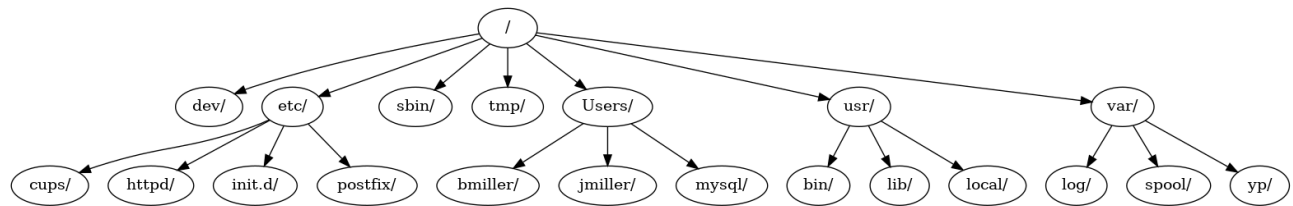
Notice that you can start at the top of the tree and follow a path made of circles and arrows all the way to the bottom. At each level of the tree we might ask ourselves a question and then follow the path that agrees with our answer.

A second property of trees is that all of the **children of one node are independent of the children of another node** while the third property is that the path to **each leaf node is unique**.

A second property of trees is that all of the **children of one node are independent of the children of another node** while the third property is that the path to **each leaf node is unique**.

Another example of a tree structure that you probably use every day is a file system. In a file system, directories, or folders, are structured as a tree. The file system tree enables you to follow a path from the root to any directory. That path will uniquely identify that subdirectory





Note that we could take the entire subtree starting with `/etc/`, detach `etc/` from the root and reattach it under `usr/`. This would change the unique pathname to `httpd` from `/etc/httpd` to `/usr/etc/httpd`, but would not affect the contents or any children of the `httpd` directory!

8.3. Vocabulary and Definitions

- Node: A node is a fundamental part of a tree. It can have a name, which we call the **key**. A node may also have additional information. We call this additional information the value or payload.

- Node: A node is a fundamental part of a tree. It can have a name, which we call the **key**. A node may also have additional information. We call this additional information the value or payload.
- Edge: An edge is another fundamental part of a tree. An edge connects two nodes to show that there is a relationship between them. Every node (except the root) is connected by **exactly one incoming edge from another node**. Each node may have several outgoing edges.

- Node: A node is a fundamental part of a tree. It can have a name, which we call the **key**. A node may also have additional information. We call this additional information the value or payload.
- Edge: An edge is another fundamental part of a tree. An edge connects two nodes to show that there is a relationship between them. Every node (except the root) is connected by **exactly one incoming edge from another node**. Each node may have several outgoing edges.
- Root: The root of the tree is the only node in the tree that has no incoming edges.
- Path: A path is an ordered list of nodes that are connected by edges, for example, Mammalia -> Carnivora -> Felidae -> Felis -> catus is a path.

- Children: The set of nodes that have incoming edges from the same node are said to be the children of that node.
- Parent: A node is the parent of all the nodes it connects to with outgoing edges.

- Children: The set of nodes that have incoming edges from the same node are said to be the children of that node.
- Parent: A node is the parent of all the nodes it connects to with outgoing edges.
- Sibling: Nodes in the tree that are children of the same parent are said to be siblings. The nodes `etc/` and `usr/` are siblings in the file system tree shown in Figure 2.
- Subtree: A subtree is a set of nodes and edges comprised of a parent and all the descendants of that parent.

- Children: The set of nodes that have incoming edges from the same node are said to be the children of that node.
- Parent: A node is the parent of all the nodes it connects to with outgoing edges.
- Sibling: Nodes in the tree that are children of the same parent are said to be siblings. The nodes `etc/` and `usr/` are siblings in the file system tree shown in Figure 2.
- Subtree: A subtree is a set of nodes and edges comprised of a parent and all the descendants of that parent.
- Leaf Node: A leaf node is a node that has no children.
- Level: The level of a node n is the number of edges on the path from the root node to n .
- Height: The height of a tree is equal to the maximum level of any node in the tree.

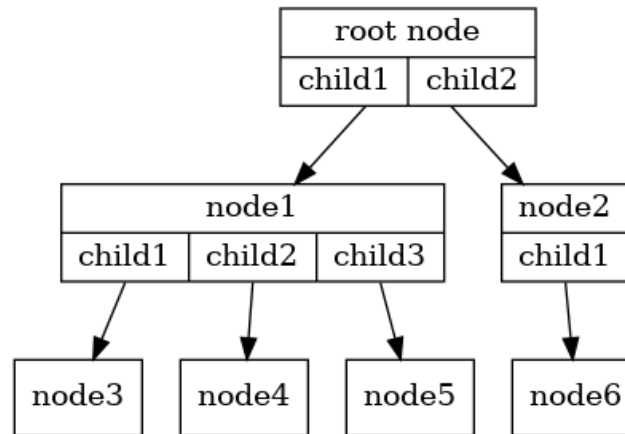
Definition One: A tree consists of a set of nodes and a set of edges that connect pairs of nodes. A tree has the following properties:

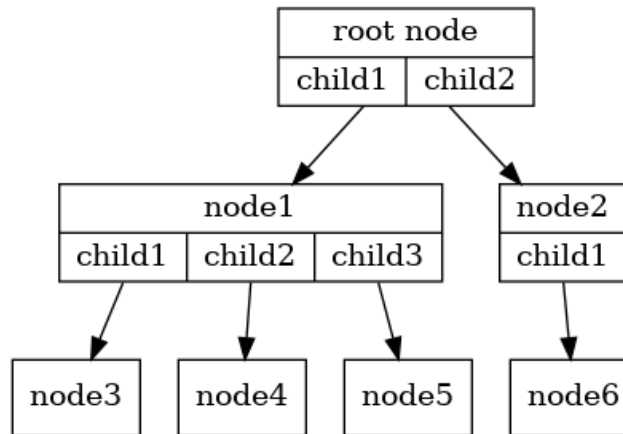
Definition One: A tree consists of a set of nodes and a set of edges that connect pairs of nodes. A tree has the following properties:

1. One node of the tree is designated as the root node.
2. Every node n , except the root node, is connected by an edge from exactly one other node p , where p is the parent of n .

Definition One: A tree consists of a set of nodes and a set of edges that connect pairs of nodes. A tree has the following properties:

1. One node of the tree is designated as the root node.
2. Every node n , except the root node, is connected by an edge from exactly one other node p , where p is the parent of n .
3. A unique path traverses from the root to each node.
4. If each node in the tree has a maximum of two children, we say that the tree is a binary tree.





The arrowheads on the edges indicate the direction of the connection.

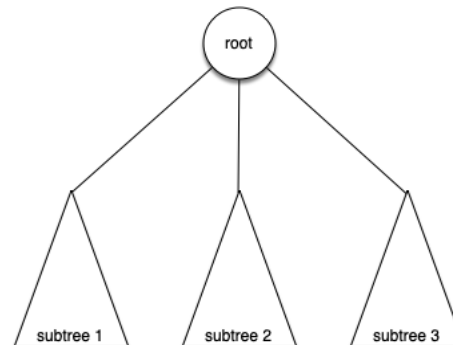
Definition Two: A tree is either empty or consists of a root and zero or more subtrees, each of which is also a tree. The root of each subtree is connected to the root of the parent tree by an edge.

Definition Two: A tree is either empty or consists of a root and zero or more subtrees, each of which is also a tree. The root of each subtree is connected to the root of the parent tree by an edge.

Figure below illustrates this recursive definition of a tree. Using the recursive definition of a tree, we know that the tree below has at least four nodes (if it is not empty) since each of the triangles representing a subtree must have a root. It may have many more nodes than that, but we do not know unless we look deeper into the tree!

Definition Two: A tree is either empty or consists of a root and zero or more subtrees, each of which is also a tree. The root of each subtree is connected to the root of the parent tree by an edge.

Figure below illustrates this recursive definition of a tree. Using the recursive definition of a tree, we know that the tree below has at least four nodes (if it is not empty) since each of the triangles representing a subtree must have a root. It may have many more nodes than that, but we do not know unless we look deeper into the tree!



```
In [ ]: display_quiz(path+"tree.json", max_width=800)
```

Which statement about edges and nodes in a tree is correct?

- ☐ Every node except the root is connected by exactly one incoming edge from another node.
- ☐ A leaf node is defined as a node that has exactly one outgoing edge.
- ☐ Every node in a tree must have at least one incoming edge.
- ☐ The height of a tree is equal to the total number of edges it contains.

Tree ADT

We can use the following functions to create and manipulate a binary tree:

We can use the following functions to create and manipulate a binary tree:

- `BinaryTree(root)` : creates a node with two `NULL` child pointers.
- `getRootVal()` / `setRootVal(value)` : read or replace the current node's value.

We can use the following functions to create and manipulate a binary tree:

- `BinaryTree(root)` : creates a node with two `NULL` child pointers.
- `getRootVal()` / `setRootVal(value)` : read or replace the current node's value.
- `getLeftChild()` and `getRightChild()` : return child pointers, or `NULL` for an empty subtree.

- `insertLeft(value)` and `insertRight(value)` : allocate and attach a new child, pushing any existing child down one level as described below.

- `insertLeft(value)` and `insertRight(value)` : allocate and attach a new child, pushing any existing child down one level as described below.

The key decision in implementing a tree is choosing a good internal storage technique. We use nodes and references, described next.

8.4. Nodes and References

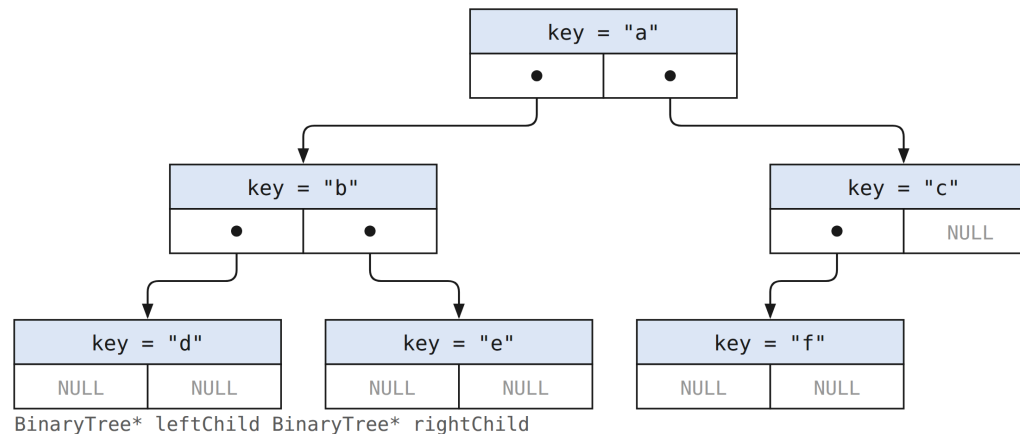
To represent a tree with nodes and references, we define a class that has attributes for the root value as well as the left and right subtrees.

To represent a tree with nodes and references, we define a class that has attributes for the root value as well as the left and right subtrees.

Using nodes and references, we might think of the tree as being structured like the one shown below:

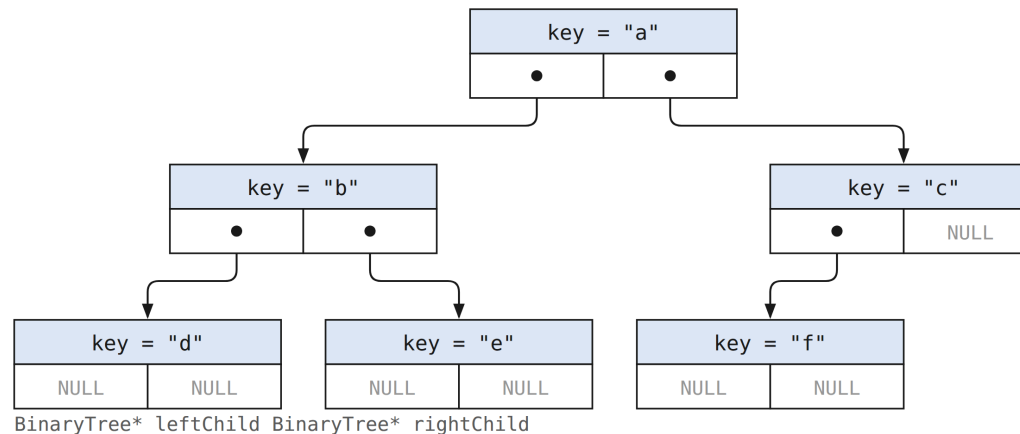
To represent a tree with nodes and references, we define a class that has attributes for the root value as well as the left and right subtrees.

Using nodes and references, we might think of the tree as being structured like the one shown below:



To represent a tree with nodes and references, we define a class that has attributes for the root value as well as the left and right subtrees.

Using nodes and references, we might think of the tree as being structured like the one shown below:



Since this representation more closely follows the object-oriented programming paradigm, we will continue to use this representation for the remainder of the chapter.

In C++, `leftChild` and `rightChild` are pointers to other `BinaryTree` objects. For example, `insertLeft()` allocates a child and updates `this->leftChild`; a `NULL` pointer represents an empty subtree.

In C++, `leftChild` and `rightChild` are pointers to other `BinaryTree` objects. For example, `insertLeft()` allocates a child and updates `this->leftChild`; a `NULL` pointer represents an empty subtree.

```
class BinaryTree {  
    public:  
        string key;  
        BinaryTree* leftChild;  
        BinaryTree* rightChild;  
  
        BinaryTree(string rootObj) {  
            key = rootObj;  
            leftChild = NULL;  
            rightChild = NULL;  
        }  
};
```

In C++, `leftChild` and `rightChild` are pointers to other `BinaryTree` objects. For example, `insertLeft()` allocates a child and updates `this->leftChild`; a `NULL` pointer represents an empty subtree.

```
class BinaryTree {  
    public:  
        string key;  
        BinaryTree* leftChild;  
        BinaryTree* rightChild;  
  
        BinaryTree(string rootObj) {  
            key = rootObj;  
            leftChild = NULL;  
            rightChild = NULL;  
        }  
};
```

A node's payload can use any chosen C++ type. In this teaching class it is a `string`; the six labels in the example therefore become six `BinaryTree` objects connected by child pointers.

To add a left child, allocate a `BinaryTree` and assign its address to `leftChild`. If a left child already exists, it becomes the new node's left child before the pointer is reassigned.

To add a left child, allocate a `BinaryTree` and assign its address to `leftChild`. If a left child already exists, it becomes the new node's left child before the pointer is reassigned.

```
void insertLeft(string newNode) {  
    if (leftChild == NULL) {  
        leftChild = new BinaryTree(newNode);  
    } else {  
        BinaryTree* newChild = new BinaryTree(newNode);  
        newChild->leftChild = leftChild;  
        leftChild = newChild;  
    }  
}
```

To add a left child, allocate a `BinaryTree` and assign its address to `leftChild`. If a left child already exists, it becomes the new node's left child before the pointer is reassigned.

```
void insertLeft(string newNode) {  
    if (leftChild == NULL) {  
        leftChild = new BinaryTree(newNode);  
    } else {  
        BinaryTree* newChild = new BinaryTree(newNode);  
        newChild->leftChild = leftChild;  
        leftChild = newChild;  
    }  
}
```

Note that we consider two cases for insertion. The first case is characterized by a node with no existing left child. When there is no left child, simply add a node to the tree. The second case is characterized by a node with an existing left child. In the second case, we insert a node and push the existing child down one level in the tree.

```
void insertRight(string newNode) {  
    if (rightChild == NULL) {  
        rightChild = new BinaryTree(newNode);  
    } else {  
        BinaryTree* newChild = new BinaryTree(newNode);  
        newChild->rightChild = rightChild;  
        rightChild = newChild;  
    }  
}
```

```
void insertRight(string newNode) {  
    if (rightChild == NULL) {  
        rightChild = new BinaryTree(newNode);  
    } else {  
        BinaryTree* newChild = new BinaryTree(newNode);  
        newChild->rightChild = rightChild;  
        rightChild = newChild;  
    }  
}
```

To round out the definition for a simple binary tree data structure, we will write accessor methods for the left and right children and for the root values:

```

void insertRight(string newNode) {
    if (rightChild == NULL) {
        rightChild = new BinaryTree(newNode);
    } else {
        BinaryTree* newChild = new BinaryTree(newNode);
        newChild->rightChild = rightChild;
        rightChild = newChild;
    }
}

```

To round out the definition for a simple binary tree data structure, we will write accessor methods for the left and right children and for the root values:

```

string getRootVal() {
    return key;
}
void setRootVal(string newKey) {
    key = newKey;
}
BinaryTree* getLeftChild() {
    return leftChild;
}
BinaryTree* getRightChild() {
    return rightChild;
}

```


Now that we have all the pieces to create and manipulate a binary tree, let's use them to check on the structure a bit more. Let's make a simple tree with node "a" as the root, and add nodes "b" and "c" as children

Now that we have all the pieces to create and manipulate a binary tree, let's use them to check on the structure a bit more. Let's make a simple tree with node "a" as the root, and add nodes "b" and "c" as children

The complete class is in `pythonds3/cppds/binarytree.hpp`. Here it is in action — note that an empty child prints as `0` (the NULL pointer):

Now that we have all the pieces to create and manipulate a binary tree, let's use them to check on the structure a bit more. Let's make a simple tree with node "a" as the root, and add nodes "b" and "c" as children

The complete class is in `pythonds3/cppds/binarytree.hpp`. Here it is in action — note that an empty child prints as `0` (the NULL pointer):

```
In [1]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp" // the class of this section
using namespace std;

int main() {
    BinaryTree aTree("a");
    cout << aTree.getRootVal() << endl;
    cout << aTree.getLeftChild() << endl;    // NULL prints as 0
    aTree.insertLeft("b");
    cout << aTree.getLeftChild()->getRootVal() << endl;
    aTree.insertRight("c");
    cout << aTree.getRightChild()->getRootVal() << endl;
    aTree.getRightChild()->setRootVal("hello");
    cout << aTree.getRightChild()->getRootVal() << endl;
    return 0;
}
```

a

0

b

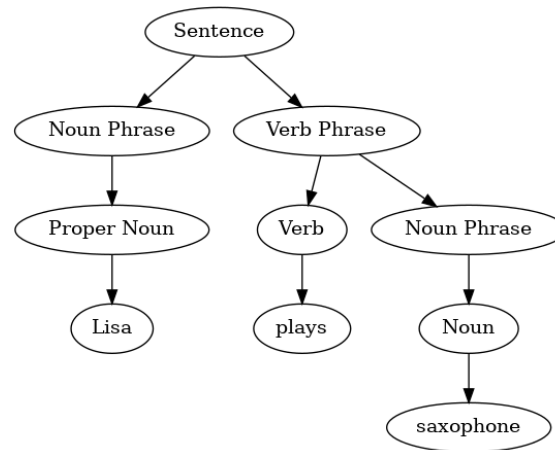
c

hello

8.5. Parse Tree

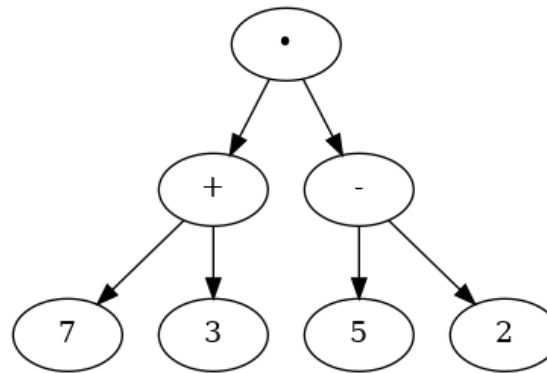
Parse trees can be used to represent real-world constructions like sentences or mathematical expressions. Figure below shows the hierarchical structure of a simple sentence. Representing a sentence as a tree structure allows us to work with the individual parts of the sentence by using subtrees.

Parse trees can be used to represent real-world constructions like sentences or mathematical expressions. Figure below shows the hierarchical structure of a simple sentence. Representing a sentence as a tree structure allows us to work with the individual parts of the sentence by using subtrees.

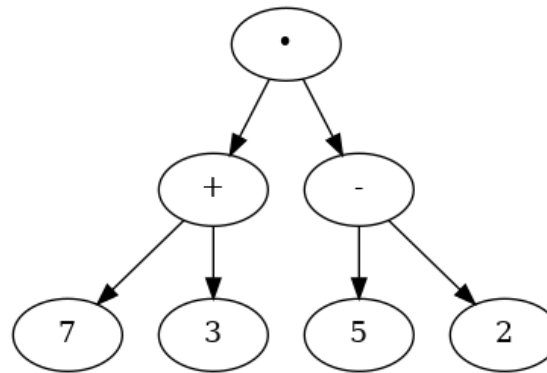


We can also represent a mathematical expression such as $((7 + 3) \cdot (5 - 2))$ as a parse tree, as shown below:

We can also represent a mathematical expression such as $((7 + 3) \cdot (5 - 2))$ as a parse tree, as shown below:



We can also represent a mathematical expression such as $((7 + 3) \cdot (5 - 2))$ as a parse tree, as shown below:



We know that multiplication has a higher precedence than either addition or subtraction. Because of the parentheses, we know that before we can do the multiplication we must evaluate the parenthesized addition and subtraction expressions. The hierarchy of the tree helps us understand the order of evaluation for the whole expression!

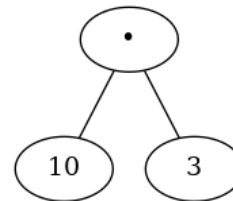
Before we can evaluate the top-level multiplication, we must evaluate the addition and the subtraction in the subtrees. The addition, which is the left subtree, evaluates to 10. The subtraction, which is the right subtree, evaluates to 3.

Before we can evaluate the top-level multiplication, we must evaluate the addition and the subtraction in the subtrees. The addition, which is the left subtree, evaluates to 10. The subtraction, which is the right subtree, evaluates to 3.

Using the hierarchical structure of trees, we can simply replace an entire subtree with one node once we have evaluated the expressions in the children. Applying this replacement procedure gives us the simplified tree shown below:

Before we can evaluate the top-level multiplication, we must evaluate the addition and the subtraction in the subtrees. The addition, which is the left subtree, evaluates to 10. The subtraction, which is the right subtree, evaluates to 3.

Using the hierarchical structure of trees, we can simply replace an entire subtree with one node once we have evaluated the expressions in the children. Applying this replacement procedure gives us the simplified tree shown below:



In the rest of this section we are going to examine parse trees in more detail. In particular we will look at

- How to build a parse tree from a fully parenthesized mathematical expression.

In the rest of this section we are going to examine parse trees in more detail. In particular we will look at

- How to build a parse tree from a fully parenthesized mathematical expression.
- How to evaluate the expression stored in a parse tree.

In the rest of this section we are going to examine parse trees in more detail. In particular we will look at

- How to build a parse tree from a fully parenthesized mathematical expression.
- How to evaluate the expression stored in a parse tree.
- How to recover the original mathematical expression from a parse tree.

In the rest of this section we are going to examine parse trees in more detail. In particular we will look at

- How to build a parse tree from a fully parenthesized mathematical expression.
- How to evaluate the expression stored in a parse tree.
- How to recover the original mathematical expression from a parse tree.

The first step in building a parse tree is to split the expression into a `vector<string>` of tokens. There are four kinds: **left parentheses**, **right parentheses**, **operators**, and **operands**.

We know that whenever we read a left parenthesis we are starting a new expression, and hence we should create a new tree to correspond to that expression. Conversely, whenever we read a right parenthesis, we have finished an expression.

We know that whenever we read a left parenthesis we are starting a new expression, and hence we should create a new tree to correspond to that expression. Conversely, whenever we read a right parenthesis, we have finished an expression.

We also know that operands are going to be leaf nodes and children of their operators. Finally, we know that every operator is going to have both a left and a right child. Using the information from above we can define four rules as follows:

We know that whenever we read a left parenthesis we are starting a new expression, and hence we should create a new tree to correspond to that expression. Conversely, whenever we read a right parenthesis, we have finished an expression.

We also know that operands are going to be leaf nodes and children of their operators. Finally, we know that every operator is going to have both a left and a right child. Using the information from above we can define four rules as follows:

1. If the current token is a `(`, add a new node as the left child of the current node, and descend to the left child.

2. If the current token is one of `{"+", "-", "/", "*"}`, store the operator in the current node, add an empty right child, and descend to that child.

2. If the current token is one of `{"+", "-", "/", "*"}`, store the operator in the current node, add an empty right child, and descend to that child.
3. If the current token is a number, set the root value of the current node to the number and return to the parent.

2. If the current token is one of `{"+", "-", "/", "*"}`, store the operator in the current node, add an empty right child, and descend to that child.
3. If the current token is a number, set the root value of the current node to the number and return to the parent.
4. If the current token is a `)`, go to the parent of the current node.

2. If the current token is one of `{"+", "-", "/", "*"}`, store the operator in the current node, add an empty right child, and descend to that child.
3. If the current token is a number, set the root value of the current node to the number and return to the parent.
4. If the current token is a `)`, go to the parent of the current node.

For $(3 + (4 * 5))$, tokenization produces the C++ sequence `{"(", "3", "+", "(", "4", "*", "5", ")", ")"}`. We begin with an empty root and apply the four rules one token at a time.

Figure below illustrates the structure and contents of the parse tree as each new token is processed:

Figure below illustrates the structure and contents of the parse tree as each new token is processed:

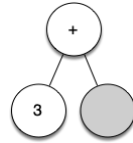


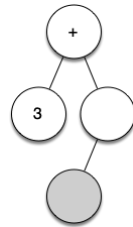
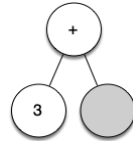
Figure below illustrates the structure and contents of the parse tree as each new token is processed:

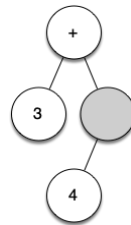
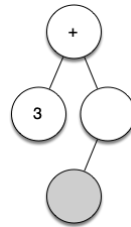
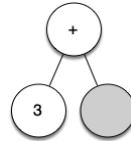


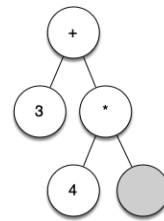
Figure below illustrates the structure and contents of the parse tree as each new token is processed:

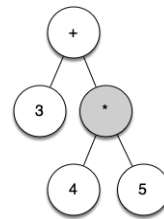
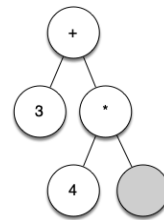












The algorithm must track the current node and its parent. Child navigation uses `getLeftChild()` and `getRightChild()` ; returning to a parent is handled by a stack.

The algorithm must track the current node and its parent. Child navigation uses `getLeftChild()` and `getRightChild()` ; returning to a parent is handled by a stack.

A simple solution to keeping track of parents as we traverse the tree is to **use a stack**. Whenever we want to descend to a child of the current node, we first push the current node on the stack. When we want to return to the parent of the current node, we pop the parent off the stack!

```
BinaryTree* buildParseTree(string fpExpr) {  
    stack<BinaryTree*> pStack;  
    BinaryTree* exprTree = new BinaryTree("");  
    pStack.push(exprTree);  
    BinaryTree* currentTree = exprTree;  
  
    stringstream ss(fpExpr);  
    string i;  
    while (ss >> i) {  
        if (i == "(") {  
            currentTree->insertLeft("");  
            pStack.push(currentTree);  
            currentTree = currentTree->getLeftChild();  
        }  
        ...  
    }
```



```

BinaryTree* buildParseTree(string fpExpr) {
    stack<BinaryTree*> pStack;
    BinaryTree* exprTree = new BinaryTree("");
    pStack.push(exprTree);
    BinaryTree* currentTree = exprTree;

    stringstream ss(fpExpr);
    string i;
    while (ss >> i) {
        if (i == "(") {
            currentTree->insertLeft("");
            pStack.push(currentTree);
            currentTree = currentTree->getLeftChild();
        }
        ...

        ...
        else if (i == "+" || i == "-" || i == "*" || i == "/") {
            currentTree->setRootVal(i);
            currentTree->insertRight("");
            pStack.push(currentTree);
            currentTree = currentTree->getRightChild();
        } else if (i == ")") {
            currentTree = pStack.top();
            pStack.pop();
        } else {
            currentTree->setRootVal(i);    // a number
            currentTree = pStack.top();
            pStack.pop();
        }
    }
}

```



```
}  
    return exprTree;  
}
```

(complete listing: `pythonds3/cppds/binarytree.hpp`)

The four construction rules appear as the branches of the C++ `if / else if` chain. A production parser should throw `invalid_argument` when a token is neither a parenthesis, a supported operator, nor a number.

The four construction rules appear as the branches of the C++ `if / else if` chain. A production parser should throw `invalid_argument` when a token is neither a parenthesis, a supported operator, nor a number.

```
In [2]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp" // buildParseTree (md Listing above)
using namespace std;

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    inorder(pt); // defined and explained in the next section
    cout << endl;
    return 0;
}
```

3 + 4 * 5

The four construction rules appear as the branches of the C++ `if / else if` chain. A production parser should throw `invalid_argument` when a token is neither a parenthesis, a supported operator, nor a number.

```
In [2]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp" // buildParseTree (md Listing above)
using namespace std;

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    inorder(pt);    // defined and explained in the next section
    cout << endl;
    return 0;
}
```

3 + 4 * 5

Now that we have built a parse tree, what can we do with it? As a first example, we will write a function to evaluate the parse tree and return the numerical result. To write this function, we will make use of the **hierarchical nature** of the tree.

Recall that we can replace the original tree with the simplified tree shown in above Figure. This suggests that we can write an algorithm that evaluates a parse tree by **recursively evaluating each subtree**.

Recall that we can replace the original tree with the simplified tree shown in above Figure. This suggests that we can write an algorithm that evaluates a parse tree by **recursively evaluating each subtree**.

A natural base case for recursive algorithms that operate on trees is to check for a leaf node. In a parse tree, the leaf nodes will always be operands. Since numerical objects like integers and floating points require no further interpretation, the evaluate function can simply return the value stored in the leaf node.

Recall that we can replace the original tree with the simplified tree shown in above Figure. This suggests that we can write an algorithm that evaluates a parse tree by **recursively evaluating each subtree**.

A natural base case for recursive algorithms that operate on trees is to check for a leaf node. In a parse tree, the leaf nodes will always be operands. Since numerical objects like integers and floating points require no further interpretation, the evaluate function can simply return the value stored in the leaf node.

The recursive step that moves the function toward the base case is to call evaluate on both the left and the right children of the current node. The recursive call effectively moves us down the tree, toward a leaf node.

To put the results of the two recursive calls together, we can simply apply the operator stored in the parent node to the results returned from evaluating both children. The code for a recursive evaluate function is shown below:

To put the results of the two recursive calls together, we can simply apply the operator stored in the parent node to the results returned from evaluating both children. The code for a recursive evaluate function is shown below:

```
double evaluate(BinaryTree* parseTree) {
    BinaryTree* leftChild = parseTree->getLeftChild();
    BinaryTree* rightChild = parseTree->getRightChild();

    if (leftChild != NULL && rightChild != NULL) {
        string op = parseTree->getRootVal();
        if (op == "+") return evaluate(leftChild) + evaluate(rightChild);
        if (op == "-") return evaluate(leftChild) - evaluate(rightChild);
        if (op == "*") return evaluate(leftChild) * evaluate(rightChild);
        return evaluate(leftChild) / evaluate(rightChild);
    } else {
        return stod(parseTree->getRootVal());    // a leaf: a number
    }
}
```

The C++ implementation stores the operator as a `string` and dispatches with `if/else if` (or a `switch` on a character), as shown in `evaluate()` above.

The C++ implementation stores the operator as a `string` and dispatches with `if/else if` (or a `switch` on a character), as shown in `evaluate()` above.

Finally, we will trace the `evaluate` function on the parse tree we created before. When we first call `evaluate()`, we pass the root of the entire tree as the parameter `parseTree`. Then we obtain references to the left and right children to make sure they exist. The recursive calls take place on lines 7–10.

The C++ implementation stores the operator as a `string` and dispatches with `if/else if` (or a `switch` on a character), as shown in `evaluate()` above.

Finally, we will trace the `evaluate` function on the parse tree we created before. When we first call `evaluate()`, we pass the root of the entire tree as the parameter `parseTree`. Then we obtain references to the left and right children to make sure they exist. The recursive calls take place on lines 7–10.

```
In [3]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp" // evaluate (md Listing above)
using namespace std;

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    cout << evaluate(pt) << endl;
    return 0;
}
```

8.6. Tree Traversals

Now it is time to look at some additional usage patterns for trees. These usage patterns can be divided into three commonly used patterns to **visit all the nodes in a tree**.

Now it is time to look at some additional usage patterns for trees. These usage patterns can be divided into three commonly used patterns to **visit all the nodes in a tree**.

The difference between these patterns is the order in which each node is visited. We call this visitation of the nodes a tree traversal. The three traversals we will look at are called preorder, inorder, and postorder. Let's start out by defining these three traversals more carefully, then look at some examples where these patterns are useful.

Now it is time to look at some additional usage patterns for trees. These usage patterns can be divided into three commonly used patterns to **visit all the nodes in a tree**.

The difference between these patterns is the order in which each node is visited. We call this visitation of the nodes a tree traversal. The three traversals we will look at are called preorder, inorder, and postorder. Let's start out by defining these three traversals more carefully, then look at some examples where these patterns are useful.

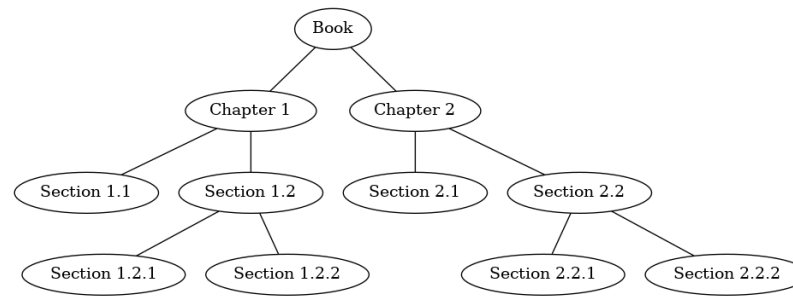
- Preorder: In a preorder traversal, we visit the root node first, then recursively do a preorder traversal of the left subtree, followed by a recursive preorder traversal of the right subtree.

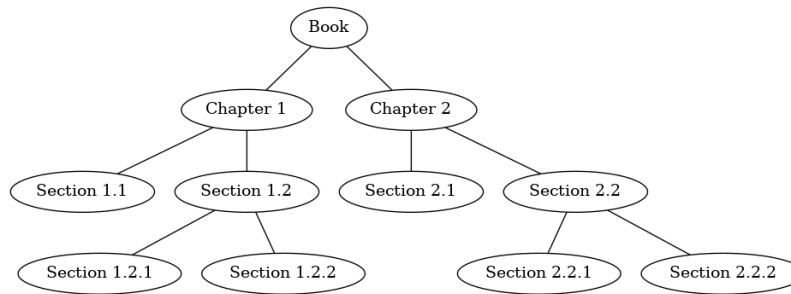
- Inorder: In an inorder traversal, we recursively do an inorder traversal on the left subtree, visit the root node, and finally do a recursive inorder traversal of the right subtree.

- Inorder: In an inorder traversal, we recursively do an inorder traversal on the left subtree, visit the root node, and finally do a recursive inorder traversal of the right subtree.
- Postorder: In a postorder traversal, we recursively do a postorder traversal of the left subtree and the right subtree followed by a visit to the root node.

- Inorder: In an inorder traversal, we recursively do an inorder traversal on the left subtree, visit the root node, and finally do a recursive inorder traversal of the right subtree.
- Postorder: In a postorder traversal, we recursively do a postorder traversal of the left subtree and the right subtree followed by a visit to the root node.

First let's look at the preorder traversal using a book as an example tree. The book is the root of the tree, and each chapter is a child of the root. Each section within a chapter is a child of the chapter, each subsection is a child of its section, and so on.





Suppose that you wanted to read this book from front to back. The preorder traversal gives you exactly that ordering.

The external preorder function is elegant because its base case is just `tree == NULL`. Otherwise it visits the root, then recursively visits the left and right subtrees.

The external preorder function is elegant because its base case is just `tree == NULL`. Otherwise it visits the root, then recursively visits the left and right subtrees.

```
void preorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        cout << tree->getRootVal() << " ";  
        preorder(tree->getLeftChild());  
        preorder(tree->getRightChild());  
    }  
}
```

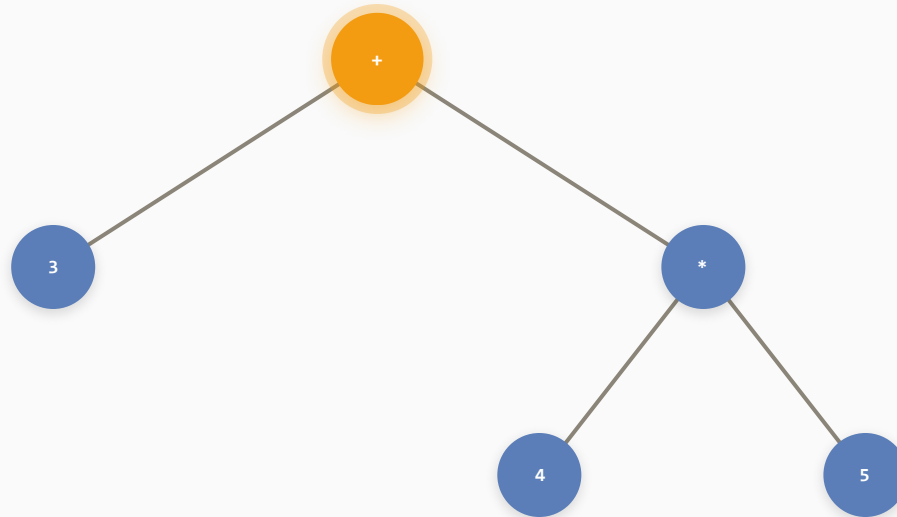
The external preorder function is elegant because its base case is just `tree == NULL`. Otherwise it visits the root, then recursively visits the left and right subtrees.

```
void preorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        cout << tree->getRootVal() << " ";  
        preorder(tree->getLeftChild());  
        preorder(tree->getRightChild());  
    }  
}
```

Output for our parse tree: `+ 3 * 4 5` — the prefix form of the expression.


```
In [ ]: load_anim("preorder")
```

前序走訪 Preorder — root → 左 → 右



› 進入 + (深度 0)

尚未訪問任何節點

解析樹 (3+(4*5))

BST 範例

完整二元樹

Implementing preorder as an external function is probably better in this case. The reason is that you very rarely want to just traverse the tree. In most cases you are going to want to accomplish something else while using one of the basic traversal patterns.

Implementing preorder as an external function is probably better in this case. The reason is that you very rarely want to just traverse the tree. In most cases you are going to want to accomplish something else while using one of the basic traversal patterns.

In fact, we will see in the next example that the postorder traversal pattern follows very closely with the code we wrote earlier to evaluate a parse tree. Therefore we will write the rest of the traversals as external functions.

Implementing preorder as an external function is probably better in this case. The reason is that you very rarely want to just traverse the tree. In most cases you are going to want to accomplish something else while using one of the basic traversal patterns.

In fact, we will see in the next example that the postorder traversal pattern follows very closely with the code we wrote earlier to evaluate a parse tree. Therefore we will write the rest of the traversals as external functions.

```
void postorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        postorder(tree->getLeftChild());  
        postorder(tree->getRightChild());  
        cout << tree->getRootVal() << " ";  
    }  
}
```

Implementing preorder as an external function is probably better in this case. The reason is that you very rarely want to just traverse the tree. In most cases you are going to want to accomplish something else while using one of the basic traversal patterns.

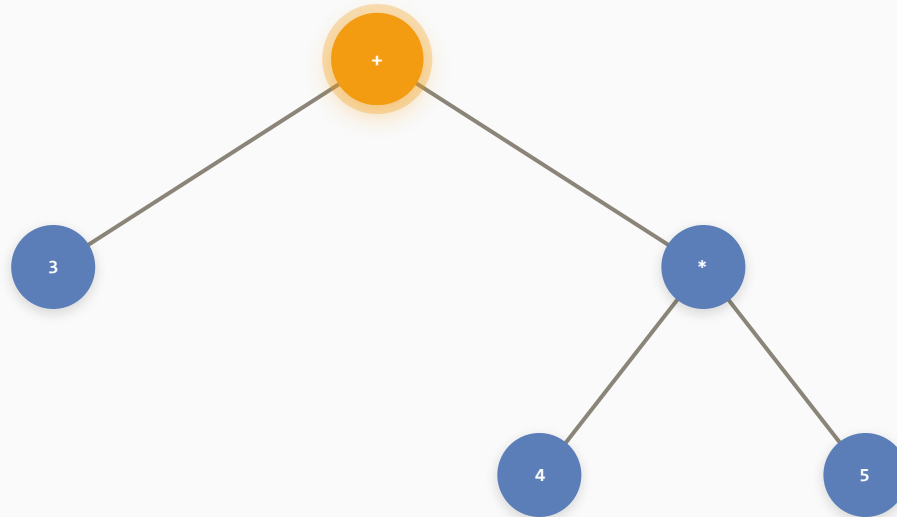
In fact, we will see in the next example that the postorder traversal pattern follows very closely with the code we wrote earlier to evaluate a parse tree. Therefore we will write the rest of the traversals as external functions.

```
void postorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        postorder(tree->getLeftChild());  
        postorder(tree->getRightChild());  
        cout << tree->getRootVal() << " ";  
    }  
}
```

Output: 3 4 5 * + — the postfix form.

```
In [ ]: load_anim("postorder")
```

後序走訪 Postorder — 左 → 右 → root



› 進入 + (深度 0)

尚未訪問任何節點

解析樹 (3+(4*5))

BST 範例

完整二元樹

We have already seen a common use for the postorder traversal, namely **evaluating a parse tree**. Assuming our binary tree is going to store only expression tree data, rewrite the evaluation function, but model it even more closely on the postorder code, we have:

We have already seen a common use for the postorder traversal, namely **evaluating a parse tree**. Assuming our binary tree is going to store only expression tree data, rewrite the evaluation function, but model it even more closely on the postorder code, we have:

```
double postordereval(BinaryTree* tree) {
    if (tree == NULL) return 0;
    if (tree->getLeftChild() != NULL && tree->getRightChild() != NULL) {
        double result1 = postordereval(tree->getLeftChild());
        double result2 = postordereval(tree->getRightChild());
        string op = tree->getRootVal();
        if (op == "+") return result1 + result2;
        if (op == "-") return result1 - result2;
        if (op == "*") return result1 * result2;
        return result1 / result2;
    }
    return stod(tree->getRootVal());
}
```


We have already seen a common use for the postorder traversal, namely **evaluating a parse tree**. Assuming our binary tree is going to store only expression tree data, rewrite the evaluation function, but model it even more closely on the postorder code, we have:

```
double postordereval(BinaryTree* tree) {
    if (tree == NULL) return 0;
    if (tree->getLeftChild() != NULL && tree->getRightChild() != NULL) {
        double result1 = postordereval(tree->getLeftChild());
        double result2 = postordereval(tree->getRightChild());
        string op = tree->getRootVal();
        if (op == "+") return result1 + result2;
        if (op == "-") return result1 - result2;
        if (op == "*") return result1 * result2;
        return result1 / result2;
    }
    return stod(tree->getRootVal());
}
```

Output: 23 — evaluation is just a post-order traversal.

Note that except that instead of printing the key at the end of the function, we return it. This allows us to save the values returned from the recursive calls. We then use these saved values along with the operator.

Note that except that instead of printing the key at the end of the function, we return it. This allows us to save the values returned from the recursive calls. We then use these saved values along with the operator.

The final traversal we will look at in this section is the inorder traversal. In the inorder traversal we visit the left subtree, followed by the root, and finally the right subtree.

Note that except that instead of printing the key at the end of the function, we return it. This allows us to save the values returned from the recursive calls. We then use these saved values along with the operator.

The final traversal we will look at in this section is the inorder traversal. In the inorder traversal we visit the left subtree, followed by the root, and finally the right subtree.

```
void inorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        inorder(tree->getLeftChild());  
        cout << tree->getRootVal() << " ";  
        inorder(tree->getRightChild());  
    }  
}
```

Note that except that instead of printing the key at the end of the function, we return it. This allows us to save the values returned from the recursive calls. We then use these saved values along with the operator.

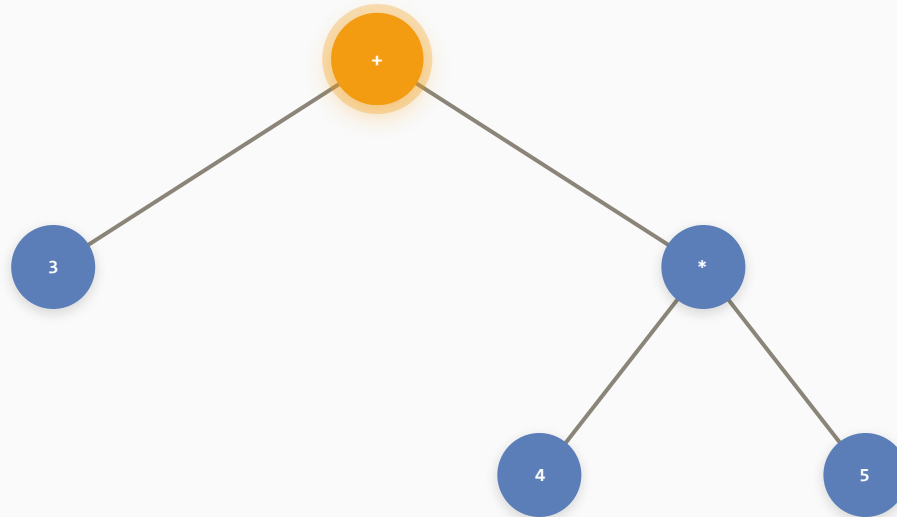
The final traversal we will look at in this section is the inorder traversal. In the inorder traversal we visit the left subtree, followed by the root, and finally the right subtree.

```
void inorder(BinaryTree* tree) {  
    if (tree != NULL) {  
        inorder(tree->getLeftChild());  
        cout << tree->getRootVal() << " ";  
        inorder(tree->getRightChild());  
    }  
}
```

Output: 3 + 4 * 5 — the familiar infix form (without parentheses).

```
In [ ]: load_anim("inorder")
```

中序走訪 Inorder — 左 → root → 右



› 進入 + (深度 0)

尚未訪問任何節點

解析樹 (3+(4*5))

BST 範例

完整二元樹

If we perform a simple inorder traversal of a parse tree, we get our original expression back without any parentheses. Let's modify the basic inorder algorithm to allow us to recover the fully parenthesized version of the expression. The only modifications we will make to the basic template are as follows.

If we perform a simple inorder traversal of a parse tree, we get our original expression back without any parentheses. Let's modify the basic inorder algorithm to allow us to recover the fully parenthesized version of the expression. The only modifications we will make to the basic template are as follows.

Print a left parenthesis before the recursive call to the left subtree, and print a right parenthesis after the recursive call to the right subtree:

If we perform a simple inorder traversal of a parse tree, we get our original expression back without any parentheses. Let's modify the basic inorder algorithm to allow us to recover the fully parenthesized version of the expression. The only modifications we will make to the basic template are as follows.

Print a left parenthesis before the recursive call to the left subtree, and print a right parenthesis after the recursive call to the right subtree:

```
string printExp(BinaryTree* tree) {  
    string result = "";  
    if (tree != NULL) {  
        result = "(" + printExp(tree->getLeftChild());  
        result = result + tree->getRootVal();  
        result = result + printExp(tree->getRightChild()) + ")";  
    }  
    return result;  
}
```


If we perform a simple inorder traversal of a parse tree, we get our original expression back without any parentheses. Let's modify the basic inorder algorithm to allow us to recover the fully parenthesized version of the expression. The only modifications we will make to the basic template are as follows.

Print a left parenthesis before the recursive call to the left subtree, and print a right parenthesis after the recursive call to the right subtree:

```
string printExp(BinaryTree* tree) {
    string result = "";
    if (tree != NULL) {
        result = "(" + printExp(tree->getLeftChild());
        result = result + tree->getRootVal();
        result = result + printExp(tree->getRightChild()) + ")";
    }
    return result;
}
```

```
In [4]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp"    // printExp (md listing above)
using namespace std;

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    cout << printExp(pt) << endl;
    return 0;
}
```

$$((3)+((4)*(5)))$$

```
In [ ]: display_quiz(path+"traversal.json", max_width=800)
```

In which type of tree traversal is the root node visited after all of its subtrees have been visited?

☐ Inorder traversal

☐ Preorder traversal

☐ Level-order traversal

☐ Postorder traversal

Exercise 1: Clean up the C++ `printExp()` function so that it does not add parentheses around individual numeric leaves.

Exercise 1: Clean up the C++ `printExp()` function so that it does not add parentheses around individual numeric leaves.

```
In [5]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp"
using namespace std;

// Modify printExp so leaf numbers are not wrapped in parentheses
string printExp2(BinaryTree* tree) {
    string result = "";
    if (tree != NULL) {
        result = "(" + printExp2(tree->getLeftChild());
        result = result + tree->getRootVal();
        result = result + printExp2(tree->getRightChild()) + ")";
    }
    return result;
}

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    cout << printExp2(pt) << endl;
    return 0;
}
```

((3)+((4)*(5)))

Exercise 1: Clean up the C++ `printExp()` function so that it does not add parentheses around individual numeric leaves.

```
In [5]: #include <iostream>
#include "pythonds3/cppds/binarytree.hpp"
using namespace std;

// Modify printExp so leaf numbers are not wrapped in parentheses
string printExp2(BinaryTree* tree) {
    string result = "";
    if (tree != NULL) {
        result = "(" + printExp2(tree->getLeftChild());
        result = result + tree->getRootVal();
        result = result + printExp2(tree->getRightChild()) + ")";
    }
    return result;
}

int main() {
    BinaryTree* pt = buildParseTree("( 3 + ( 4 * 5 ) )");
    cout << printExp2(pt) << endl;
    return 0;
}
```

`((3)+((4)*(5)))`

Goal: print `(3 + (4 * 5))` instead of `((3) + ((4) * (5)))`.

8.7. Priority Queues with Binary Heaps

A priority queue can use a **vector** in two simple ways. Keeping it sorted makes finding an extreme $O(1)$ but insertion $O(n)$ because elements shift; keeping it unsorted makes insertion $O(1)$ but finding and removing the minimum $O(n)$.

A priority queue can use a **vector** in two simple ways. Keeping it sorted makes finding an extreme $O(1)$ but insertion $O(n)$ because elements shift; keeping it unsorted makes insertion $O(1)$ but finding and removing the minimum $O(n)$.

We can do better. The classic way to implement a priority queue is using a data structure called a binary heap. A binary heap will allow us both to enqueue and dequeue items in $O(\log n)$.

A priority queue can use a `vector` in two simple ways. Keeping it sorted makes finding an extreme $O(1)$ but insertion $O(n)$ because elements shift; keeping it unsorted makes insertion $O(1)$ but finding and removing the minimum $O(n)$.

We can do better. The classic way to implement a priority queue is using a data structure called a binary heap. A binary heap will allow us both to enqueue and dequeue items in $O(\log n)$.

A binary heap offers a better balance. It looks like a complete binary tree but is stored in one `vector`; in a min-heap the smallest key is at index 0. Both insertion and removal are $O(\log n)$, while reading the minimum is $O(1)$.

8.8. Priority Queues with Binary Heaps Example

Suppose an operating system schedules jobs (priority, name) as (2, compile), (5, backup), (1, interrupt), and (3, render). A min-heap returns interrupt first, then compile, render, and backup, without fully sorting the remaining jobs. The heap promises only that the minimum pair is at the root; ties can be broken by the job key.

8.9. Binary Heap Operations

- `BinaryHeap()` : creates an empty min-heap.
- `insert(k)` : inserts a key in $O(\log n)$.
- `findMin()` : returns the minimum without removing it in $O(1)$.

- `BinaryHeap()` : creates an empty min-heap.
- `insert(k)` : inserts a key in $O(\log n)$.
- `findMin()` : returns the minimum without removing it in $O(1)$.
- `delMin()` : removes and returns the minimum in $O(\log n)$; legacy course code may call the same operation `delete()`.
- `isEmpty()` and `size()` : report heap state.
- `buildHeap(values)` : bottom-up $O(n)$ construction; `heapify(values)` is retained as a compatible legacy name.

```
In [6]: #include <iostream>
#include "pythonds3/cppds/binaryheap.hpp"    // the class of this section
using namespace std;

int main() {
    BinaryHeap myHeap;
    myHeap.insert(5);
    myHeap.insert(7);
    myHeap.insert(3);
    myHeap.insert(11);

    cout << myHeap.delet() << endl;
    cout << myHeap.delet() << endl;
    cout << myHeap.delet() << endl;
    cout << myHeap.delet() << endl;
    return 0;
}
```

3

5

7

11

```
In [6]: #include <iostream>
#include "pythonds3/cppds/binaryheap.hpp"    // the class of this section
using namespace std;

int main() {
    BinaryHeap myHeap;
    myHeap.insert(5);
    myHeap.insert(7);
    myHeap.insert(3);
    myHeap.insert(11);

    cout << myHeap.delete() << endl;
    cout << myHeap.delete() << endl;
    cout << myHeap.delete() << endl;
    cout << myHeap.delete() << endl;
    return 0;
}
```

3

5

7

11

Notice that no matter what order we add items to the heap, the smallest is removed each time!

8.10. Binary Heap Implementation

In order to make our heap work efficiently, we will take advantage of the logarithmic nature of the binary tree to represent our heap.

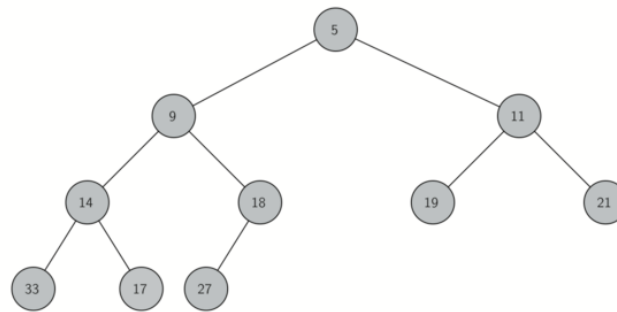
In order to make our heap work efficiently, we will take advantage of the logarithmic nature of the binary tree to represent our heap.

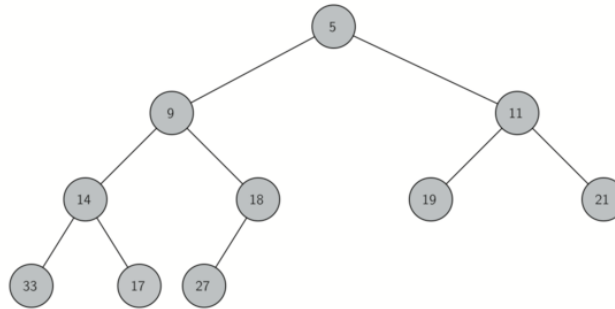
In order to guarantee logarithmic performance, we must keep our tree **balanced**. A balanced binary tree has roughly the same number of nodes in the left and right subtrees of the root. In our heap implementation we keep the tree balanced by creating a complete binary tree.

In order to make our heap work efficiently, we will take advantage of the logarithmic nature of the binary tree to represent our heap.

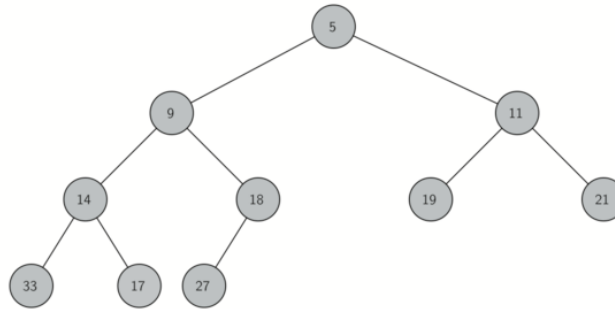
In order to guarantee logarithmic performance, we must keep our tree **balanced**. A balanced binary tree has roughly the same number of nodes in the left and right subtrees of the root. In our heap implementation we keep the tree balanced by creating a complete binary tree.

A complete binary tree is a tree in which each level has all of its nodes. The exception to this is the bottom level of the tree, which we fill in from left to right. Figure below shows an example of a complete binary tree.





A complete tree can be stored in one **vector** instead of using node pointers. With zero-based indexing, the left child of index p is at $2p + 1$.



A complete tree can be stored in one `vector` instead of using node pointers. With zero-based indexing, the left child of index p is at $2p + 1$.

Similarly, the right child of index p is at $2p + 2$. With zero-based indexing, the parent of a non-root node at index i is computed by integer division as $(i - 1) / 2$ in C++.

Figure below shows a complete binary tree and also gives the vector representation of the tree. Note the $2p + 1$ and $2p + 2$ relationship between parent and children.

Figure below shows a complete binary tree and also gives the vector representation of the tree. Note the $2p + 1$ and $2p + 2$ relationship between parent and children.

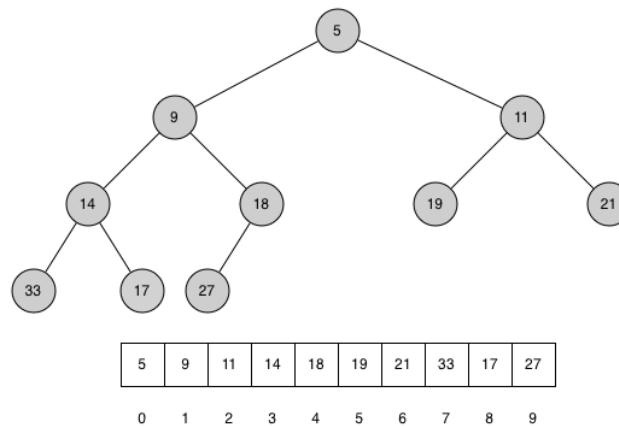
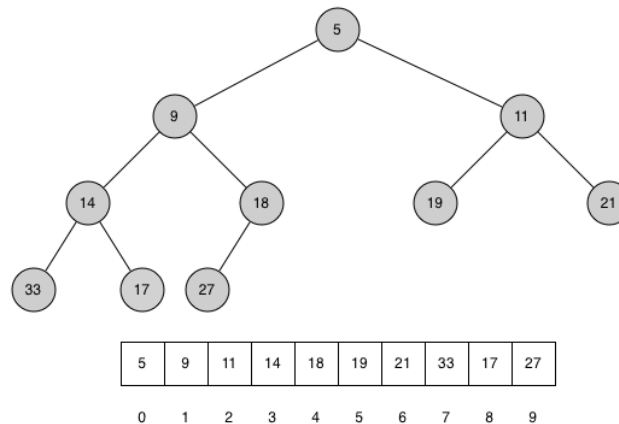


Figure below shows a complete binary tree and also gives the vector representation of the tree. Note the $2p + 1$ and $2p + 2$ relationship between parent and children.



The vector representation of the tree, along with the full structure property, allows us to efficiently traverse a complete binary tree using only a few simple mathematical operations.

The method that we will use to store items in a heap relies on maintaining the heap order property. The heap order property is as follows: in a heap, for every node x with parent p , the key in p is smaller than or equal to the key in x . Figure above also illustrates a complete binary tree that has the heap order property.

The method that we will use to store items in a heap relies on maintaining the heap order property. The heap order property is as follows: in a heap, for every node x with parent p , the key in p is smaller than or equal to the key in x . Figure above also illustrates a complete binary tree that has the heap order property.

A heap needs only one data member, `vector<int> heap`, because the complete-tree relationships are encoded by zero-based indices.

The method that we will use to store items in a heap relies on maintaining the heap order property. The heap order property is as follows: in a heap, for every node x with parent p , the key in p is smaller than or equal to the key in x . Figure above also illustrates a complete binary tree that has the heap order property.

A heap needs only one data member, `vector<int> heap`, because the complete-tree relationships are encoded by zero-based indices.

```
class BinaryHeap {  
    public:  
        vector<int> heap;    // the complete tree, stored flat  
  
        BinaryHeap() {}  
        bool isEmpty() {  
            return heap.empty();  
        }  
};
```

With the root at index 0, node i has children at $2i + 1$ and $2i + 2$, and its parent at $(i - 1) / 2$.

To insert, append the item to the end of the **vector**, preserving the complete-tree **structure**. Appending may violate the heap **order** property, so the new item then percolates upward.

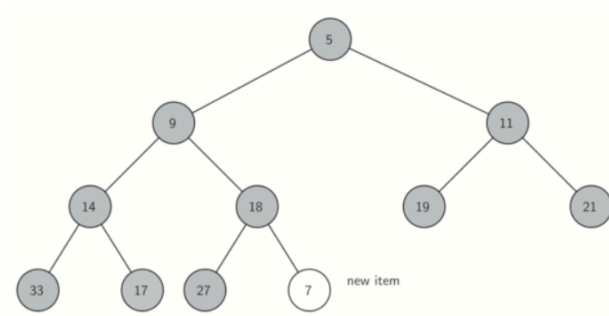
To insert, append the item to the end of the `vector`, preserving the complete-tree **structure**. Appending may violate the heap **order** property, so the new item then percolates upward.

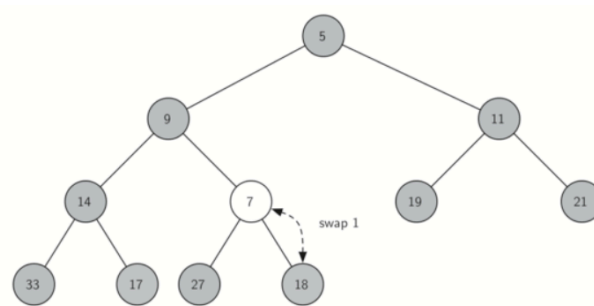
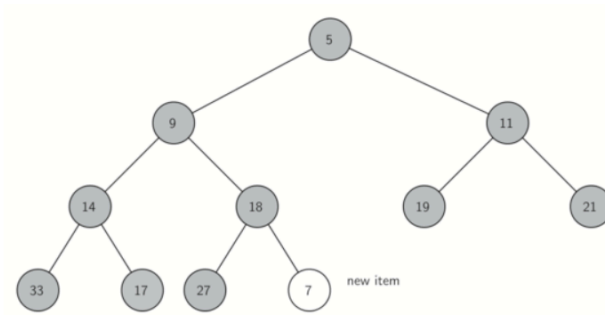
However, it is possible to write a method that will allow us to regain the heap structure property by comparing the newly added item with its parent. If the newly added item is less than its parent, then we can swap the item with its parent.

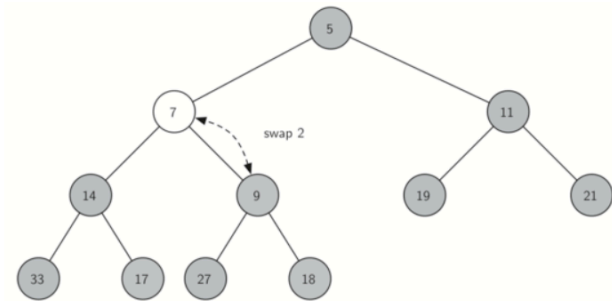
To insert, append the item to the end of the `vector`, preserving the complete-tree **structure**. Appending may violate the heap **order** property, so the new item then percolates upward.

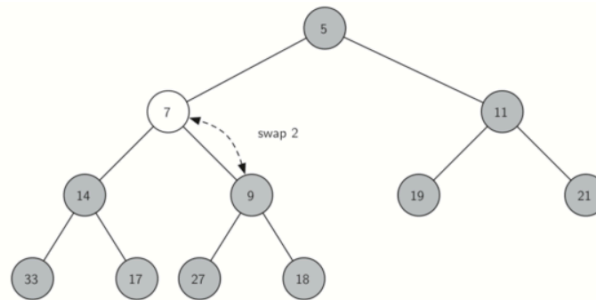
However, it is possible to write a method that will allow us to regain the heap structure property by comparing the newly added item with its parent. If the newly added item is less than its parent, then we can swap the item with its parent.

Figure below shows the series of swaps needed to percolate the newly added item up to its proper position in the tree.









Notice that when we percolate an item up, we are restoring the heap property between the newly added item and the parent. We are also preserving the heap property for any siblings. Of course, if the newly added item is very small, we may still need to swap it up another level. In fact, we may need to keep swapping until we get to the top of the tree!

```
void percup(int i) {  
    while (i > 0) {  
        int parentIdx = (i - 1) / 2;  
        if (heap[i] < heap[parentIdx]) {  
            swap(heap[i], heap[parentIdx]);  
        } else {  
            break;  
        }  
        i = parentIdx;  
    }  
}
```

```

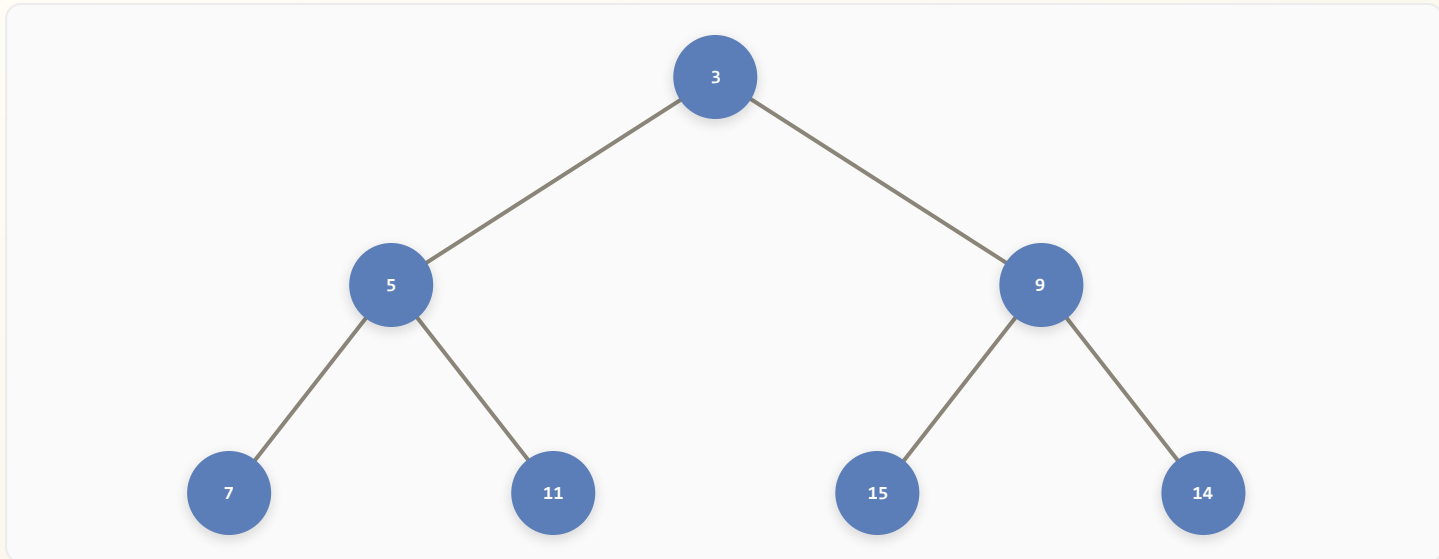
void percup(int i) {
    while (i > 0) {
        int parentIdx = (i - 1) / 2;
        if (heap[i] < heap[parentIdx]) {
            swap(heap[i], heap[parentIdx]);
        } else {
            break;
        }
        i = parentIdx;
    }
}

```

Most of `insert()` is handled by `percUp()`: append the item, then compare it with ancestors until heap order is restored.

```
In [ ]: load_anim("heap_insert")
```

Min-Heap Insert — perc_up 動畫



陣列表示 (heap[i]) :

0	1	2	3	4	5	6
3	5	9	7	11	15	14

› 輸入 key 後按「插入」或「開始」

插入 key :

2

插入

當前 heap :

3, 5, 9, 7, 11, 15, 14

套用

```
void insert(int item) {  
    heap.push_back(item);  
    percUp(heap.size() - 1);  
}
```

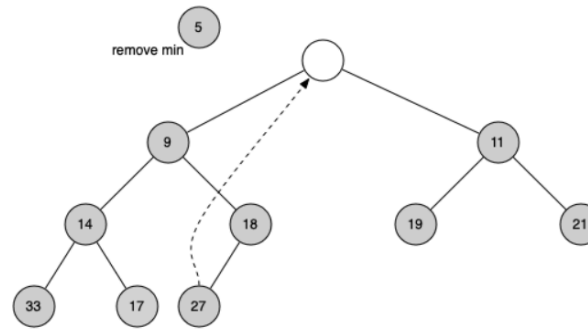
```
void insert(int item) {  
    heap.push_back(item);  
    percUp(heap.size() - 1);  
}
```

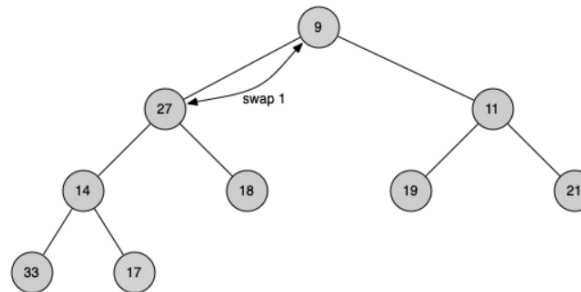
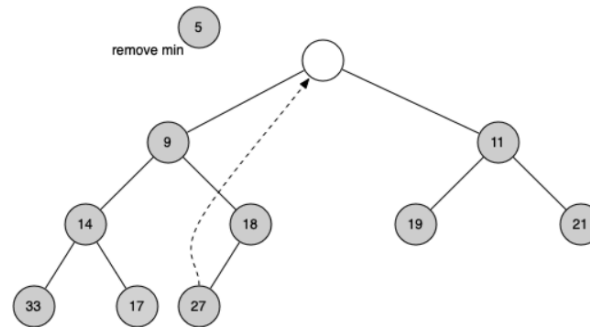
For a min-heap, `findMin()` simply reads `heap[0]`. The `delMin()` operation (legacy name `delete()`) removes that value and restores both heap properties.

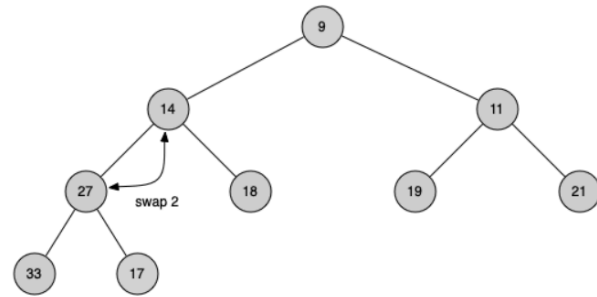
```
void insert(int item) {  
    heap.push_back(item);  
    percUp(heap.size() - 1);  
}
```

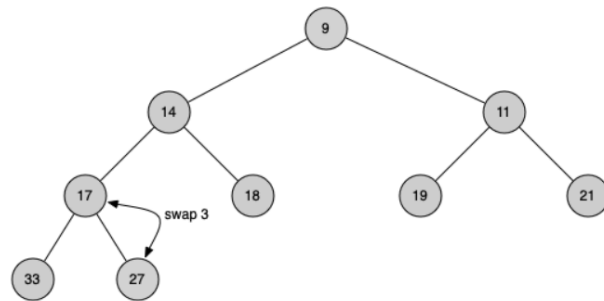
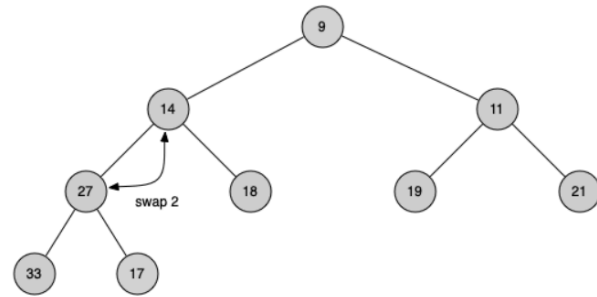
For a min-heap, `findMin()` simply reads `heap[0]`. The `delMin()` operation (legacy name `delete()`) removes that value and restores both heap properties.

The hard part of `delMin()` is restoring the complete-tree structure and heap order after removing the root. It does this in two steps.









In order to maintain the heap order property, all we need to do is swap the root with its smaller child that is less than the root. After the initial swap, we may repeat the swapping process with a node and its children until the node is swapped into a position on the tree where it is already less than both children.

In order to maintain the heap order property, all we need to do is swap the root with its smaller child that is less than the root. After the initial swap, we may repeat the swapping process with a node and its children until the node is swapped into a position on the tree where it is already less than both children.

```
void percDown(int i) {
    while (2 * i + 1 < (int)heap.size()) {
        int smChild = getMinChild(i);
        if (heap[i] > heap[smChild]) {
            swap(heap[i], heap[smChild]);
        } else {
            break;
        }
        i = smChild;
    }
}

int getMinChild(int i) {
    if (2 * i + 2 > (int)heap.size() - 1) {
        return 2 * i + 1;
    }
    if (heap[2 * i + 1] < heap[2 * i + 2]) {
        return 2 * i + 1;
    }
    return 2 * i + 2;
}
```

After the last vector item replaces the root, `percDown()` repeatedly swaps it with its smaller child until heap order is restored.

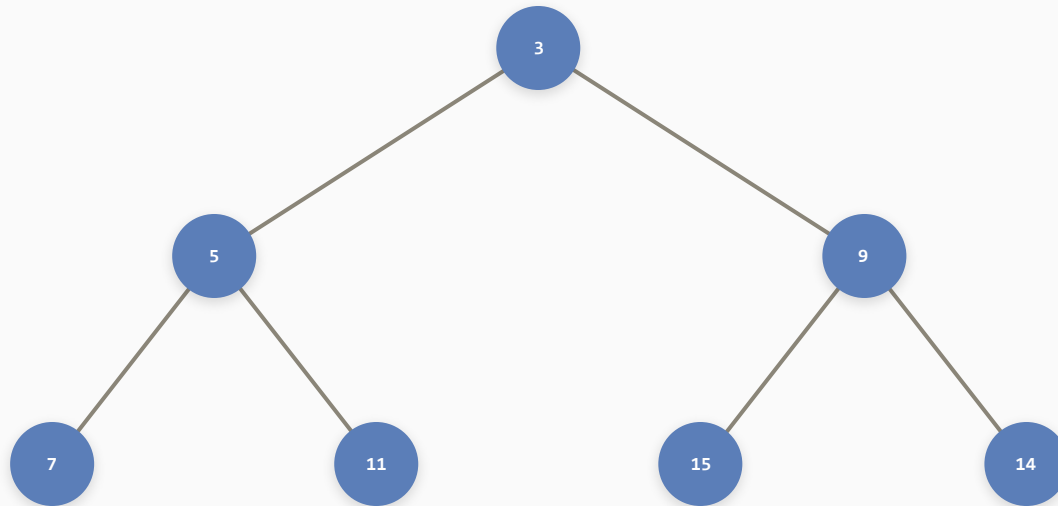
After the last vector item replaces the root, `percDown()` repeatedly swaps it with its smaller child until heap order is restored.

```
int delet() { // legacy name; delete is a C++ keyword
    if (heap.empty()) throw underflow_error("empty heap");
    swap(heap[0], heap.back());
    int result = heap.back();
    heap.pop_back();
    if (!heap.empty()) percDown(0);
    return result;
}

int delMin() { return delet(); }
```

```
In [ ]: load_anim("heap_extract")
```

Min-Heap Delete-min — perc_down 動畫



陣列表示 (heap[i]) :

0	1	2	3	4	5	6
3	5	9	7	11	15	14

› 按「開始」執行 delete-min

插入 key :

?

插入

當前 heap :

3, 5, 9, 7, 11, 15, 14

套用

One construction method inserts the n keys one at a time. `insert()` appends and may percolate through the heap height, so each insertion is $O(\log n)$ in the worst case.

One construction method inserts the n keys one at a time. `insert()` appends and may percolate through the heap height, so each insertion is $O(\log n)$ in the worst case.

Repeated insertion therefore builds a heap in $O(n \log n)$ time. It does **not** insert into the middle of a sorted vector; heap order is only a parent/child condition.

One construction method inserts the n keys one at a time. `insert()` appends and may percolate through the heap height, so each insertion is $O(\log n)$ in the worst case.

Repeated insertion therefore builds a heap in $O(n \log n)$ time. It does **not** insert into the middle of a sorted vector; heap order is only a parent/child condition.

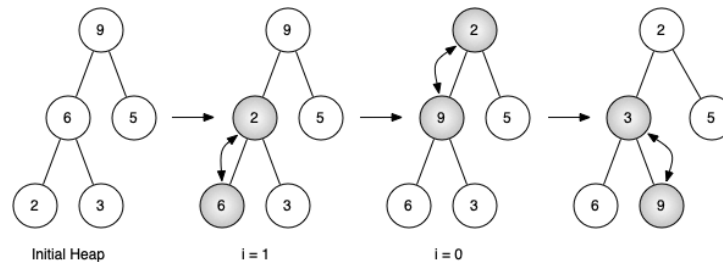
Bottom-up `buildHeap()` (legacy name `heapify()`) is faster: start at the last non-leaf index and call `percDown()` moving toward the root. Its total cost is $O(n)$.

```
void heapify(vector<int> notAHeap) {  
    heap = notAHeap;    // copy the vector  
    int i = heap.size() / 2 - 1;    // last non-leaf node  
    while (i >= 0) {  
        percDown(i);  
        i = i - 1;  
    }  
}
```

```

void heapify(vector<int> notAHeap) {
    heap = notAHeap;    // copy the vector
    int i = heap.size() / 2 - 1;    // last non-leaf node
    while (i >= 0) {
        percDown(i);
        i = i - 1;
    }
}

```



The figure traces `heapify()` on `[9, 6, 5, 2, 3]`. Starting at the last non-leaf node, `percDown()` swaps a violating parent with its **smaller child**. Indices at or beyond `heap.size()/2` are leaves and need no work.

The figure traces `heapify()` on `[9, 6, 5, 2, 3]`. Starting at the last non-leaf node, `percDown()` swaps a violating parent with its **smaller child**. Indices at or beyond `heap.size()/2` are leaves and need no work.

When `i = 0`, percolating from the root may cross several levels. `percDown()` checks the smaller child again after each swap, so 9 moves as far downward as necessary.

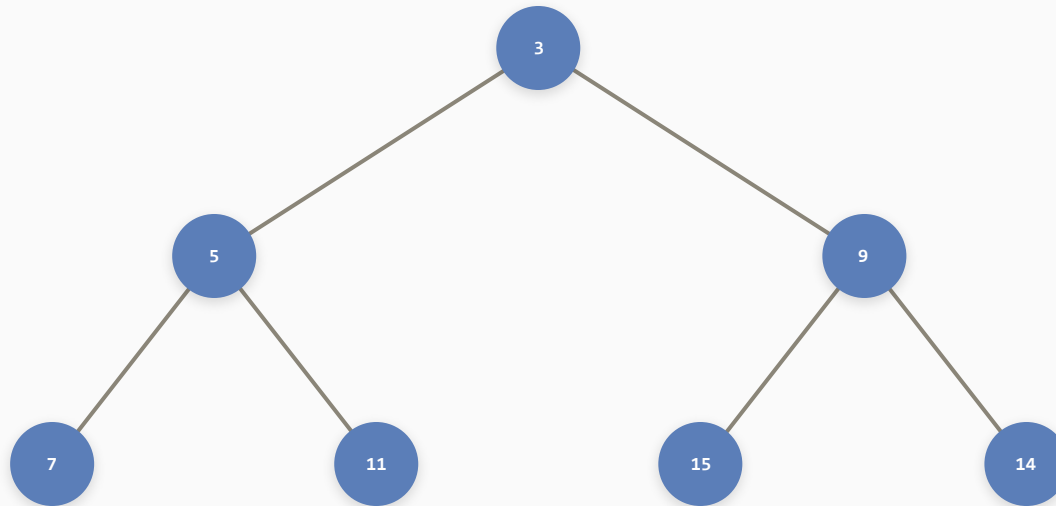
The figure traces `heapify()` on `[9, 6, 5, 2, 3]`. Starting at the last non-leaf node, `percDown()` swaps a violating parent with its **smaller child**. Indices at or beyond `heap.size()/2` are leaves and need no work.

When `i = 0`, percolating from the root may cross several levels. `percDown()` checks the smaller child again after each swap, so 9 moves as far downward as necessary.

In this case it results in a second swap with 3. Now that 9 has been moved to the lowest level of the tree, no further swapping can be done.


```
In [ ]: load_anim("heap_build")
```

Heapify — 從未排序陣列建堆積 ($O(n)$)



陣列表示 (heap[i]) :

0	1	2	3	4	5	6
3	5	9	7	11	15	14

› 陣列當作未排序輸入, 按「開始」做 heapify

插入 key :

2

插入

當前 heap :

9. 5. 3. 7. 11. 1. 14

套用

```
In [8]: #include <iostream>
#include "pythonds3/cppds/binaryheap.hpp"
using namespace std;

int main() {
    BinaryHeap aHeap;
    aHeap.heapify({10, 4, 9, 8, 12, 15, 3, 5, 14, 18});
    aHeap.print();
    return 0;
}
```

3 4 9 5 12 15 10 8 14 18

```
In [8]: #include <iostream>
#include "pythonds3/cppds/binaryheap.hpp"
using namespace std;

int main() {
    BinaryHeap aHeap;
    aHeap.heapify({10, 4, 9, 8, 12, 15, 3, 5, 14, 18});
    aHeap.print();
    return 0;
}
```

3 4 9 5 12 15 10 8 14 18

The $O(n)$ bound follows by charging work by node height: many nodes are leaves and cost zero, about $n/4$ nodes move at most one level, $n/8$ at most two, and so on. Thus $\sum_{h \geq 0} (n/2^{h+1})h = O(n)$, not $O(n \log n)$.

```
In [ ]: display_quiz(path+"binary_heap.json", max_width=800)
```

What is the primary reason for using a Binary Heap to implement a Priority Queue?

- ☐ It eliminates the need for any comparison operations during insertion.
- ☐ It allows us to find the maximum or minimum element in $O(n)$ time.
- ☐ Its complete-tree shape keeps height $O(\log n)$, supporting insertion and delete-min in $O(\log n)$.
- ☐ It stores elements in a completely sorted order in the underlying list.

How do repeated insertion and bottom-up buildHeap compare when constructing a heap from n keys?

- ☐ buildHeap must fully sort the vector first.
- ☐ Repeated insertion is $O(n^2)$, while buildHeap is $O(\log n)$.
- ☐ Both always take $O(n)$.
- ☐ Repeated insertion is $O(n \log n)$, while bottom-up buildHeap is $O(n)$.

Exercise 2: Using `buildHeap()` (or legacy `heapify()`) and `delMin()` (or legacy `delet()`), write a function that sorts a vector in $O(n \log n)$ time.

Exercise 2: Using `buildHeap()` (or legacy `heapify()`) and `delMin()` (or legacy `delet()`), write a function that sorts a vector in $O(n \log n)$ time.

In [9]:

```
#include <iostream>
#include <vector>
#include "pythonds3/cppds/binaryheap.hpp"
using namespace std;

vector<int> heapSort(vector<int> unsortedList) {
    BinaryHeap heap;
    vector<int> sortedList;
    ____;           // 1. build the heap in O(n)
    while (____) {   // 2. repeatedly delete the minimum, O(log n) each
        ____;
    }
    return sortedList;
}

int main() {
    for (int x : heapSort({10, 3, 5, 1, 15, 7, 9, 2, 8})) cout << x << " ";
    return 0;
}
```

/tmp/tmp8i16vm1q.cpp: In function 'std::vector<int> heapSort(std::vector<int>)':

/tmp/tmp8i16vm1q.cpp:11:5: error: '____' was not declared in this scope

```
11 | ____;
   |    ^~~~
```

```
[C++ kernel] Error: Unable to compile the source code. Return error: 0  
x1.
```


Exercise 2: Using `buildHeap()` (or legacy `heapify()`) and `delMin()` (or legacy `delet()`), write a function that sorts a vector in $O(n \log n)$ time.

In [9]:

```
#include <iostream>
#include <vector>
#include "pythonds3/cppds/binaryheap.hpp"
using namespace std;

vector<int> heapSort(vector<int> unsortedList) {
    BinaryHeap heap;
    vector<int> sortedList;
    ____;           // 1. build the heap in O(n)
    while (____) {   // 2. repeatedly delete the minimum, O(log n) each
        ____;
    }
    return sortedList;
}

int main() {
    for (int x : heapSort({10, 3, 5, 1, 15, 7, 9, 2, 8})) cout << x << " ";
    return 0;
}
```

/tmp/tmp8i16vm1q.cpp: In function 'std::vector<int> heapSort(std::vector<int>)':

/tmp/tmp8i16vm1q.cpp:11:5: error: '____' was not declared in this scope

```
11 | ____;
    | ^~~~
```

```
[C++ kernel] Error: Unable to compile the source code. Return error: 0x1.
```

Available `BinaryHeap` methods: `insert`, `findMin`, `delMin`, `size`, `buildHeap`, plus compatible legacy names `delet` and `heapify`, and the helpers `isEmpty`, `percUp`, `percDown`, `getMinChild`, `print`. Empty `findMin` / `delMin` calls throw `underflow_error`.

Expected output: `Sorted list: 1 2 3 5 7 8 9 10 15`

8.11. Binary Search Trees

We have already seen two map implementations: binary search on a sorted vector and a hash table. A binary search tree provides a third representation for key-value pairs.

We have already seen two map implementations: binary search on a sorted vector and a hash table. A binary search tree provides a third representation for key-value pairs.

In this section we will study binary search trees as yet another way to map from a key to a value. In this case we are not interested in the exact placement of items in the tree, but we are interested in using the binary tree structure to provide for efficient searching.

8.12. Search Tree Operations

Before the implementation, review the C++ course interface:

- `BinarySearchTree()` : creates an empty map.
- `put(key, value)` : inserts a pair, or replaces the value if the key already exists.
- `get(key)` : returns the stored value (the simplified string implementation uses `""` for missing keys).

Before the implementation, review the C++ course interface:

- `BinarySearchTree()` : creates an empty map.
- `put(key, value)` : inserts a pair, or replaces the value if the key already exists.
- `get(key)` : returns the stored value (the simplified string implementation uses `""` for missing keys).
- `remove(key)` : removes a key-value pair and throws `invalid_argument` if it is absent.
- `length()` : returns the number of distinct keys.
- `contains(key)` : returns a C++ `bool`.

These named member functions provide the complete Map ADT interface.

8.13. Search Tree Implementation

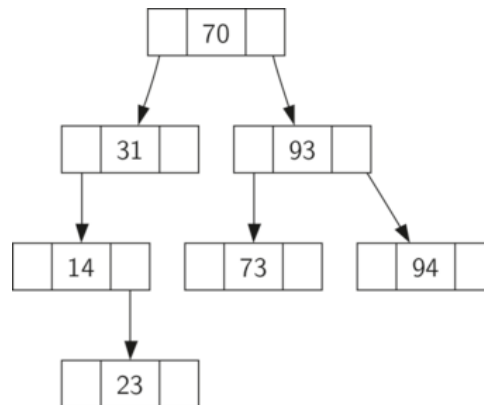
A binary search tree (BST) relies on the property that keys that are less than the parent are found in the left subtree, and keys that are greater than the parent are found in the right subtree. We will call this the BST property.

A binary search tree (BST) relies on the property that keys that are less than the parent are found in the left subtree, and keys that are greater than the parent are found in the right subtree. We will call this the BST property.

Figure below illustrates this property of a binary search tree, showing the keys without any associated values.

A binary search tree (BST) relies on the property that keys that are less than the parent are found in the left subtree, and keys that are greater than the parent are found in the right subtree. We will call this the BST property.

Figure below illustrates this property of a binary search tree, showing the keys without any associated values.



Now that you know what a binary search tree is, we will look at how a binary search tree is constructed. The search tree above represents the nodes that exist after we have inserted the following keys in the order shown: 70, 31, 93, 94, 14, 23, 73. Since 70 was the first key inserted into the tree, it is the root.

Now that you know what a binary search tree is, we will look at how a binary search tree is constructed. The search tree above represents the nodes that exist after we have inserted the following keys in the order shown: 70, 31, 93, 94, 14, 23, 73. Since 70 was the first key inserted into the tree, it is the root.

Next, 31 is less than 70, so it becomes the left child of 70. Next, 93 is greater than 70, so it becomes the right child of 70. Now we have two levels of the tree filled, so the next key is going to be the left or right child of either 31 or 93.

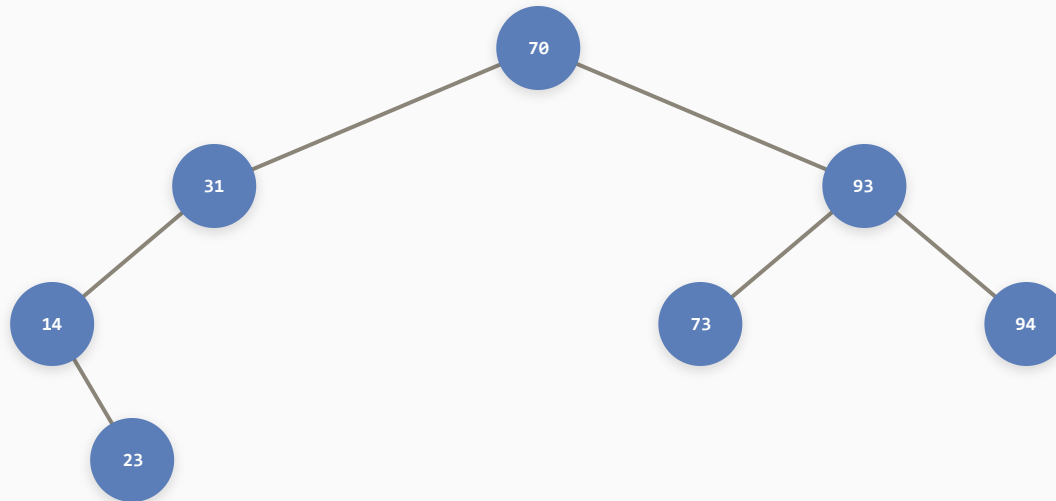
Now that you know what a binary search tree is, we will look at how a binary search tree is constructed. The search tree above represents the nodes that exist after we have inserted the following keys in the order shown: 70, 31, 93, 94, 14, 23, 73. Since 70 was the first key inserted into the tree, it is the root.

Next, 31 is less than 70, so it becomes the left child of 70. Next, 93 is greater than 70, so it becomes the right child of 70. Now we have two levels of the tree filled, so the next key is going to be the left or right child of either 31 or 93.

Since 94 is greater than 70 and 93, it becomes the right child of 93. Similarly 14 is less than 70 and 31, so it becomes the left child of 31. 23 is also less than 31, so it must be in the left subtree of 31. However, it is greater than 14, so it becomes the right child of 14.


```
In [ ]: load_anim("bst_search")
```

BST 搜尋 (get) — 沿樹下移比較 key



已建立 BST (7 個 keys)

初始 keys : 70, 31, 93, 94, 14, 23, 73

建立樹

隨機

搜尋 key : 23

get (搜尋)

→ 單步

↺ 重置

速度

To implement the binary search tree, we will use the **nodes and references** approach similar to the one we used to implement the linked list and the expression tree.

To implement the binary search tree, we will use the **nodes and references** approach similar to the one we used to implement the linked list and the expression tree.

However, because we must be able create and work with a binary search tree that is empty, our implementation will use two classes. The first class we will call `BinarySearchTree` , and the second class we will call `TreeNode` .

To implement the binary search tree, we will use the **nodes and references** approach similar to the one we used to implement the linked list and the expression tree.

However, because we must be able create and work with a binary search tree that is empty, our implementation will use two classes. The first class we will call `BinarySearchTree`, and the second class we will call `TreeNode`.

The `BinarySearchTree` class has a reference to the `TreeNode` that is the root of the binary search tree. In most cases the external methods defined in the outer class simply check to see if the tree is empty. If there are nodes in the tree, the request is just passed on to a private method defined in the `BinarySearchTree` class that takes the root as a parameter.

In the case where the tree is empty or we want to delete the key at the root of the tree, we must take special action.

In the case where the tree is empty or we want to delete the key at the root of the tree, we must take special action.

```
class BinarySearchTree {
public:
    TreeNode* root = NULL;
    int size = 0;

    BinarySearchTree() = default;
    ~BinarySearchTree();           // releases every node
    BinarySearchTree(const BinarySearchTree& other); // deep copy
    int length() const { return size; }
};
```

The complete header also supplies copy/move assignment so two trees never own the same nodes.

In the case where the tree is empty or we want to delete the key at the root of the tree, we must take special action.

```
class BinarySearchTree {
public:
    TreeNode* root = NULL;
    int size = 0;

    BinarySearchTree() = default;
    ~BinarySearchTree();           // releases every node
    BinarySearchTree(const BinarySearchTree& other); // deep copy
    int length() const { return size; }
};
```

The complete header also supplies copy/move assignment so two trees never own the same nodes.

The `TreeNode` class provides many helper methods that make the work done in the `BinarySearchTree` class methods much easier.

```
class TreeNode {
    public:
        string key;
        string value;
        TreeNode* leftChild;
        TreeNode* rightChild;
        TreeNode* parent;

        TreeNode(string k, string v, TreeNode* p = NULL) {
            key = k;
            value = v;
            leftChild = NULL;
            rightChild = NULL;
            parent = p;
        }
    ...
}
```



```

class TreeNode {
public:
    string key;
    string value;
    TreeNode* leftChild;
    TreeNode* rightChild;
    TreeNode* parent;

    TreeNode(string k, string v, TreeNode* p = NULL) {
        key = k;
        value = v;
        leftChild = NULL;
        rightChild = NULL;
        parent = p;
    }
    ...

    ...
    bool isLeftChild() {
        return parent != NULL && parent->leftChild == this;
    }
    bool isRightChild() {
        return parent != NULL && parent->rightChild == this;
    }
    bool isLeaf() {
        return leftChild == NULL && rightChild == NULL;
    }
    bool hasAnyChild() {
        return leftChild != NULL || rightChild != NULL;
    }
}

```

```
bool hasBothChildren() {  
    return leftChild != NULL && rightChild != NULL;  
}  
};
```

Each `TreeNode` stores a parent pointer. It is especially useful when `remove()` reconnects a child or splices out an inorder successor.

Each `TreeNode` stores a parent pointer. It is especially useful when `remove()` reconnects a child or splices out an inorder successor.

The constructor's default parent argument lets us create either a root (`parent == NULL`) or a child linked to its parent.

Each `TreeNode` stores a parent pointer. It is especially useful when `remove()` reconnects a child or splices out an inorder successor.

The constructor's default parent argument lets us create either a root (`parent == NULL`) or a child linked to its parent.

Now that we have the `BinarySearchTree` shell and the `TreeNode`, it is time to write the `put()` method that will allow us to build our binary search tree. The method is a method of the `BinarySearchTree` class. This method will check to see if the tree already has a root. If there is not a root, then `put()` will create a new `TreeNode` and install it as the root of the tree.

`put()` delegates every insertion, including creation of the root, to the protected virtual `insertOrAssign()` hook. A balanced-tree subclass can override that hook without bypassing the public `put()` API or its distinct-key `size` accounting.

`put()` delegates every insertion, including creation of the root, to the protected virtual `insertOrAssign()` hook. A balanced-tree subclass can override that hook without bypassing the public `put()` API or its distinct-key `size` accounting.

- Starting at the root of the tree, search the binary tree comparing the new key to the key in the current node. If the new key is less than the current node, search the left subtree. If the new key is greater than the current node, search the right subtree.

`put()` delegates every insertion, including creation of the root, to the protected virtual `insertOrAssign()` hook. A balanced-tree subclass can override that hook without bypassing the public `put()` API or its distinct-key `size` accounting.

- Starting at the root of the tree, search the binary tree comparing the new key to the key in the current node. If the new key is less than the current node, search the left subtree. If the new key is greater than the current node, search the right subtree.
- When there is no left or right child to search, we have found the position in the tree where the new node should be installed.
- To add a node to the tree, create a new `TreeNode` object and insert the object at the point discovered in the previous step.

```

protected:
virtual bool insertOrAssign(string key, string value,
                           TreeNode*& slot, TreeNode* parent) {
    if (slot == NULL) {
        slot = new TreeNode(key, value, parent);
        return true;
    }
    if (key == slot->key) {
        slot->value = value;           // duplicate: update, do not grow size
        return false;
    }
    TreeNode*& child = (key < slot->key) ? slot->leftChild : slot-
>rightChild;
    return insertOrAssign(key, value, child, slot);
}

public:
void put(string key, string value) {
    if (insertOrAssign(key, value, root, NULL)) size++;
}

```

The legacy public `_put(key, value, currentNode)` wrapper remains available, but subclasses extend insertion through the protected virtual hook.

```

protected:
virtual bool insertOrAssign(string key, string value,
                           TreeNode*& slot, TreeNode* parent) {
    if (slot == NULL) {
        slot = new TreeNode(key, value, parent);
        return true;
    }
    if (key == slot->key) {
        slot->value = value;           // duplicate: update, do not grow size
        return false;
    }
    TreeNode*& child = (key < slot->key) ? slot->leftChild : slot-
>rightChild;
    return insertOrAssign(key, value, child, slot);
}

public:
void put(string key, string value) {
    if (insertOrAssign(key, value, root, NULL)) size++;
}

```

The legacy public `_put(key, value, currentNode)` wrapper remains available, but subclasses extend insertion through the protected virtual hook.

The course header keeps the interface explicit: call `put(key, value)`. An `operator[]` overload could be added in a more general template, but it is not part of this teaching implementation.

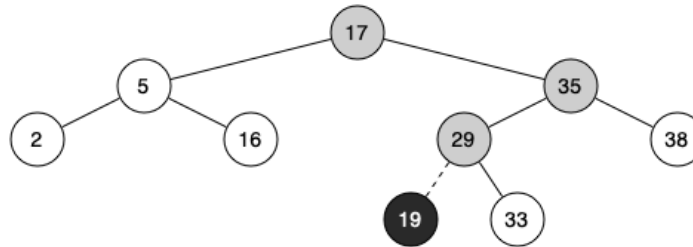
For example, use `myTree.put("a", "apple")`, then `myTree.get("a")`. This is ordinary C++ method syntax and matches the header used by the executable cells.

For example, use `myTree.put("a", "apple")`, then `myTree.get("a")`. This is ordinary C++ method syntax and matches the header used by the executable cells.

Figure below illustrates the process for inserting a new node into a binary search tree. The lightly shaded nodes indicate the nodes that were visited during the insertion process.

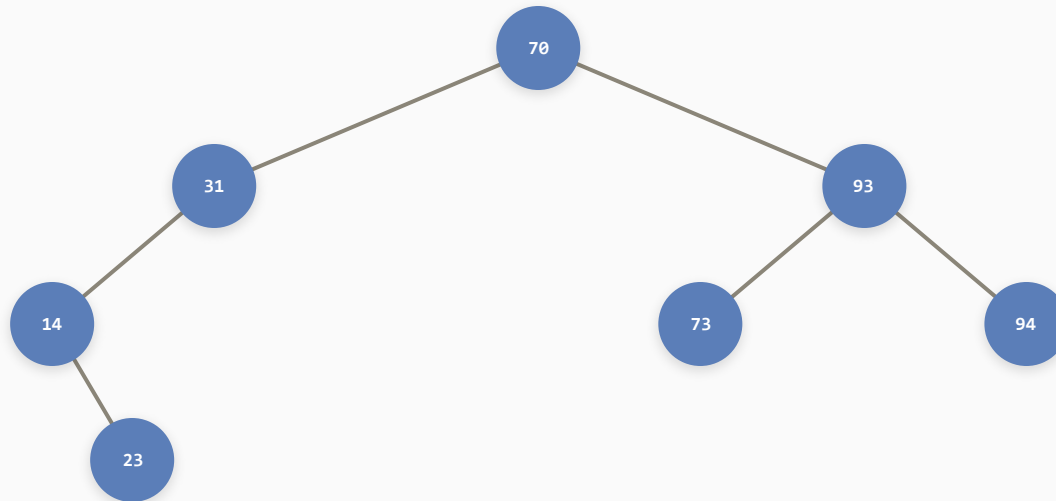
For example, use `myTree.put("a", "apple")`, then `myTree.get("a")`. This is ordinary C++ method syntax and matches the header used by the executable cells.

Figure below illustrates the process for inserting a new node into a binary search tree. The lightly shaded nodes indicate the nodes that were visited during the insertion process.



```
In [ ]: load_anim("bst_insert")
```

BST 插入 (put) — 沿 BST 性質找空位



已建立 BST (7 個 keys)

初始 keys : 70, 31, 93, 94, 14, 23, 73

建立樹

隨機

插入 key :

40

put (插入)

單步

重置

速度

The `get()` method is even easier than the `put()` method because it simply searches the tree recursively until it gets to a non-matching leaf node or finds a matching key. When a matching key is found, the value stored in the payload of the node is returned.

The `get()` method is even easier than the `put()` method because it simply searches the tree recursively until it gets to a non-matching leaf node or finds a matching key. When a matching key is found, the value stored in the payload of the node is returned.

```
string get(string key) {
    if (root != NULL) {
        TreeNode* result = _get(key, root);
        if (result != NULL) {
            return result->value;
        }
    }
    return "";
}

TreeNode* _get(string key, TreeNode* currentNode) {
    if (currentNode == NULL) {
        return NULL;
    }
    if (currentNode->key == key) {
        return currentNode;
    } else if (key < currentNode->key) {
        return _get(key, currentNode->leftChild);
    } else {
        return _get(key, currentNode->rightChild);
    }
}
```

Membership testing is a direct use of `_get()`:

Membership testing is a direct use of `_get()`:

```
bool contains(string key) {  
    return _get(key, root) != NULL;  
}
```

Finally, we turn our attention to the most challenging operation on the binary search tree, the deletion of a key. The first task is to find the node to delete by searching the tree. If the tree has more than one node we search using the `_get()` method to find the `TreeNode` that needs to be removed.

Finally, we turn our attention to the most challenging operation on the binary search tree, the deletion of a key. The first task is to find the node to delete by searching the tree. If the tree has more than one node we search using the `_get()` method to find the `TreeNode` that needs to be removed.

If the tree only has a single node, that means we are removing the root of the tree, but we still must check to make sure the key of the root matches the key that is to be deleted!

Finally, we turn our attention to the most challenging operation on the binary search tree, the deletion of a key. The first task is to find the node to delete by searching the tree. If the tree has more than one node we search using the `_get()` method to find the `TreeNode` that needs to be removed.

If the tree only has a single node, that means we are removing the root of the tree, but we still must check to make sure the key of the root matches the key that is to be deleted!

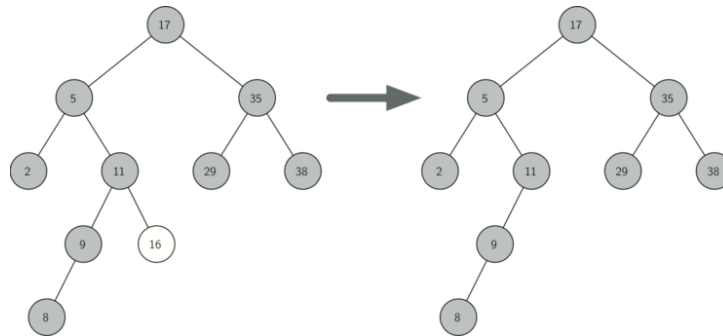
```
void remove(string key) {
    TreeNode* nodeToRemove = _get(key, root);
    if (nodeToRemove == NULL)
        throw invalid_argument("Error, key not in tree");
    _delete(nodeToRemove);           // reconnects links and deletes
storage
    size--;
}
```

The root case is handled inside `_delete()`; removing the last node sets `root` to `NULL` and frees it.

Once we've found the node containing the key we want to delete, there are three cases that we must consider:

Once we've found the node containing the key we want to delete, there are three cases that we must consider:

1. The node to be deleted has no children



The first case is straightforward. If the current node has no children, all we need to do is delete the node and remove the reference to this node in the parent.

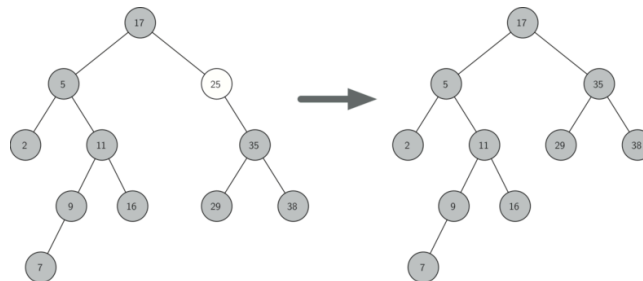
The first case is straightforward. If the current node has no children, all we need to do is delete the node and remove the reference to this node in the parent.

```
if (currentNode->isLeaf()) {    // removing a leaf
    if (currentNode == currentNode->parent->leftChild) {
        currentNode->parent->leftChild = NULL;
    } else {
        currentNode->parent->rightChild = NULL;
    }
    delete currentNode;
}
```

The first case is straightforward. If the current node has no children, all we need to do is delete the node and remove the reference to this node in the parent.

```
if (currentNode->isLeaf()) {    // removing a leaf
    if (currentNode == currentNode->parent->leftChild) {
        currentNode->parent->leftChild = NULL;
    } else {
        currentNode->parent->rightChild = NULL;
    }
    delete currentNode;
}
```

2. The node to be deleted has only one child



The second case is only slightly more complicated. If a node has only a single child, then we can simply promote the child to take the place of its parent. Since the cases are symmetric with respect to either having a left or right child, we will just discuss the case where the current node has a left child.

The second case is only slightly more complicated. If a node has only a single child, then we can simply promote the child to take the place of its parent. Since the cases are symmetric with respect to either having a left or right child, we will just discuss the case where the current node has a left child.

1. If the current node is a left child, then we only need to update the parent reference of the left child to point to the parent of the current node, and then update the left child reference of the parent to point to the current node's left child.

The second case is only slightly more complicated. If a node has only a single child, then we can simply promote the child to take the place of its parent. Since the cases are symmetric with respect to either having a left or right child, we will just discuss the case where the current node has a left child.

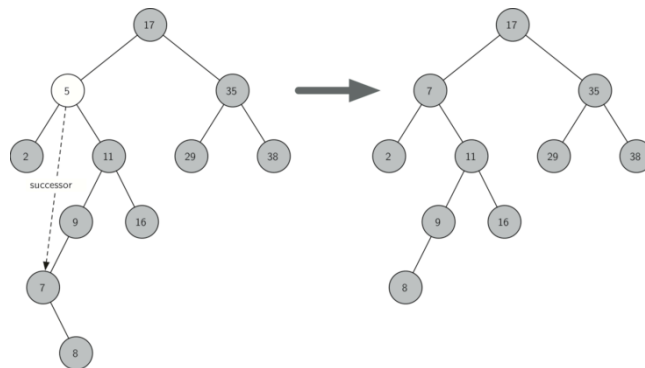
1. If the current node is a left child, then we only need to update the parent reference of the left child to point to the parent of the current node, and then update the left child reference of the parent to point to the current node's left child.
2. If the node is a right child, connect its parent directly to the node's only child and update that child's parent pointer.
3. If the node is the root, set `root` to its only child (or `NULL`) and clear the new root's parent pointer. In every case, delete the removed node afterward.

```

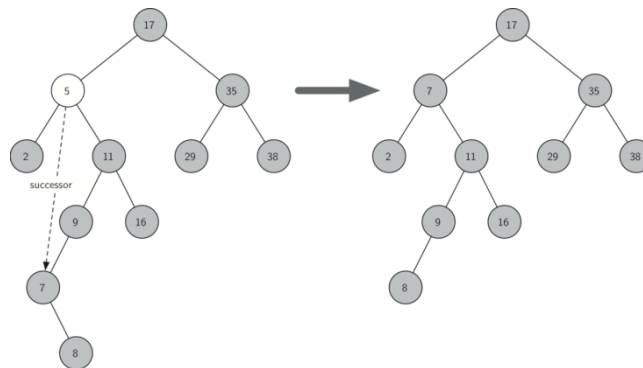
else {    // removing a node with one child
    if (currentNode->leftChild != NULL) {
        if (currentNode->isLeftChild()) {
            currentNode->leftChild->parent = currentNode->parent;
            currentNode->parent->leftChild = currentNode->leftChild;
        } else if (currentNode->isRightChild()) {
            currentNode->leftChild->parent = currentNode->parent;
            currentNode->parent->rightChild = currentNode->leftChild;
        }
    }
    // (mirror image for a right child)
    delete currentNode;
}

```

3. The node to be deleted has two children



3. The node to be deleted has two children



The third case is the most difficult case to handle. If a node has two children, then it is unlikely that we can simply promote one of them to take the node's place!

We can, however, search the tree for a node that can be used to replace the one scheduled for deletion. What we need is a node that will preserve the BST relationships for both of the existing left and right subtrees. The node that will do this is the node that has the next-largest key in the tree. We call this node the successor.

We can, however, search the tree for a node that can be used to replace the one scheduled for deletion. What we need is a node that will preserve the BST relationships for both of the existing left and right subtrees. The node that will do this is the node that has the next-largest key in the tree. We call this node the successor.

The successor has at most one child, so `spliceOut()` reconnects that child, the target receives the successor's key/value, and then the successor node is deleted. The C++ methods are `findSuccessor()` and `spliceOut()`.

We can, however, search the tree for a node that can be used to replace the one scheduled for deletion. What we need is a node that will preserve the BST relationships for both of the existing left and right subtrees. The node that will do this is the node that has the next-largest key in the tree. We call this node the successor.

The successor has at most one child, so `spliceOut()` reconnects that child, the target receives the successor's key/value, and then the successor node is deleted. The C++ methods are `findSuccessor()` and `spliceOut()`.

```
else if (currentNode->hasBothChildren()) {    // removing a node with two
children
    TreeNode* successor = currentNode->findSuccessor();
    successor->spliceOut();
    currentNode->key = successor->key;
    currentNode->value = successor->value;
    delete successor;
}
```

For deletion we only ever need the simple case — the node has a right subtree, so the successor is the minimum of that subtree:

```
TreeNode* findSuccessor() {  
    return rightChild->findMin();  
}
```

For deletion we only ever need the simple case — the node has a right subtree, so the successor is the minimum of that subtree:

```
TreeNode* findSuccessor() {  
    return rightChild->findMin();  
}
```

This code makes use of the same properties of binary search trees that cause an **inorder traversal** to print out the nodes in the tree from smallest to largest. In general, there are three cases to consider when looking for the successor:

1. If the node has a right child, then the successor is the smallest key in the right subtree.

For deletion we only ever need the simple case — the node has a right subtree, so the successor is the minimum of that subtree:

```
TreeNode* findSuccessor() {  
    return rightChild->findMin();  
}
```

This code makes use of the same properties of binary search trees that cause an **inorder traversal** to print out the nodes in the tree from smallest to largest. In general, there are three cases to consider when looking for the successor:

1. If the node has a right child, then the successor is the smallest key in the right subtree.
2. If the node has no right child and is the left child of its parent, then the parent is the successor.
3. If the node is the right child of its parent, and itself has no right child, then the successor to this node is the successor of its parent, excluding this node.

But here we only care about the first one since we have children!

```
TreeNode* findMin() {    // a method of the TreeNode class
    TreeNode* current = this;
    while (current->leftChild != NULL) {
        current = current->leftChild;
    }
    return current;
}
```



```
TreeNode* findMin() { // a method of the TreeNode class
    TreeNode* current = this;
    while (current->leftChild != NULL) {
        current = current->leftChild;
    }
    return current;
}
```

`findMin()` follows `leftChild` pointers until the next one is `NULL`; that leftmost node has the smallest key in the subtree.

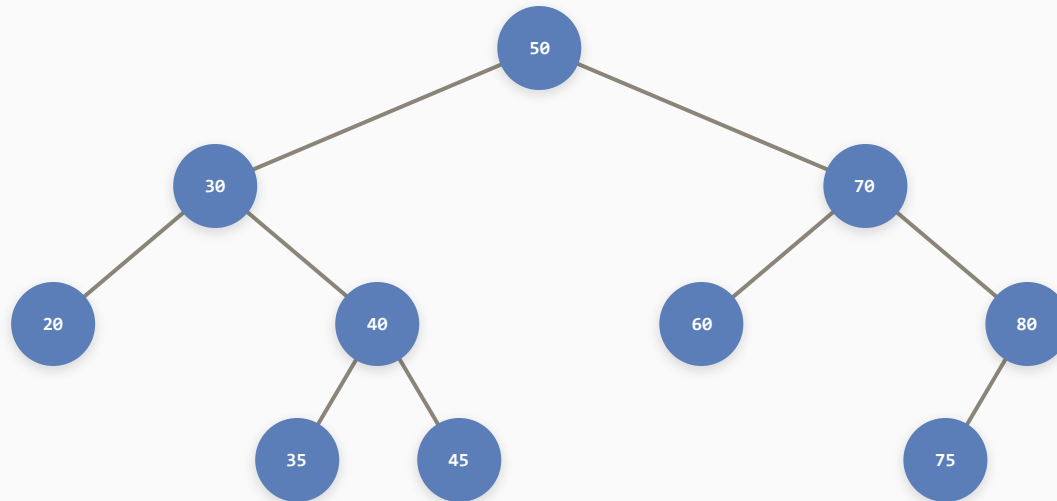
```

void spliceOut() {    // a method of the TreeNode class
    if (isLeaf()) {
        if (isLeftChild()) {
            parent->leftChild = NULL;
        } else {
            parent->rightChild = NULL;
        }
    } else if (hasAnyChild()) {
        TreeNode* child = (leftChild != NULL) ? leftChild : rightChild;
        if (isLeftChild()) {
            parent->leftChild = child;
        } else {
            parent->rightChild = child;
        }
        child->parent = parent;
    }
}

```

```
In [ ]: load_anim("bst_delete")
```

BST 刪除 — 三種情境與 in-order successor



已建立 BST (10 個 keys) 。選擇 key 並按「刪除」

樹結構 (依序插入): 50, 30, 70, 20, 40, 60, 80, 35, 45, 75

建立樹

刪除 key:

30

▶ 刪除

→ 單步

↺ 重建

Case 1: 刪 20 (葉)

Case 2: 刪 80 (一子)

Case 3: 刪 30 (兩子)

Case 3: 刪 50 (root)

To process every BST key in order, use the inorder traversal already developed. The compact course class prints values directly; a reusable version can append keys to an output vector.

To process every BST key in order, use the inorder traversal already developed. The compact course class prints values directly; a reusable version can append keys to an output vector.

C++ in-order traversal is a recursive function that visits keys in sorted order:

```
void inorder(TreeNode* node) {  
    if (node != NULL) {  
        inorder(node->leftChild);  
        cout << node->value << " ";  
        inorder(node->rightChild);  
    }  
}
```

To process every BST key in order, use the inorder traversal already developed. The compact course class prints values directly; a reusable version can append keys to an output vector.

C++ in-order traversal is a recursive function that visits keys in sorted order:

```
void inorder(TreeNode* node) {  
    if (node != NULL) {  
        inorder(node->leftChild);  
        cout << node->value << " ";  
        inorder(node->rightChild);  
    }  
}
```

The recursion is explicit: each call processes a smaller subtree, and the base case is a `NULL` pointer. Appending during the middle visit produces the BST keys in sorted order.

```

In [10]: #include <iostream>
#include "pythonds3/cppds/bst.hpp"    // TreeNode + BinarySearchTree (md Listing
using namespace std;

int main() {
    BinarySearchTree myTree;
    myTree.put("a", "a");           myTree.put("q", "quick");
    myTree.put("b", "brown");       myTree.put("f", "fox");
    myTree.put("j", "jumps");       myTree.put("o", "over");
    myTree.put("t", "the");          myTree.put("l", "lazy");
    myTree.put("d", "dog");

    cout << myTree.get("q") << " " << myTree.get("l") << endl;
    cout << "There are " << myTree.length() << " items in this tree" << endl;
    myTree.remove("a");
    cout << "There are " << myTree.length() << " items in this tree" << endl;
    myTree.inorder(myTree.root);
    cout << endl;
    return 0;
}

```

quick lazy

There are 9 items in this tree

There are 8 items in this tree

brown dog fox jumps lazy over quick the

Exercise 3: Use `put()` and in-order traversal to sort a vector in $O(n \log n)$ average time. State why the worst case is $O(n^2)$ for an unbalanced BST.

Exercise 3: Use `put()` and in-order traversal to sort a vector in $O(n \log n)$ average time. State why the worst case is $O(n^2)$ for an unbalanced BST.

In [11]:

```
#include <iostream>
#include <vector>
#include "pythonds3/cppds/bst.hpp"
using namespace std;

vector<string> treeSort(vector<string> values) {
    BinarySearchTree bst;
    vector<string> result;
    ____; // 1. insert all elements (O(log n) each on average)
    ____; // 2. in-order traversal visits keys in sorted order (O(n))
    return result;
}

int main() {
    for (string s : treeSort({"t", "a", "o", "j", "d", "b", "q", "f", "l"}))
        cout << s << " ";
    return 0;
}
```

```
/tmp/tmp95kpdu0k.cpp: In function 'std::vector<std::__cxx11::basic_string<char> > treeSort(std::vector<std::__cxx11::basic_string<char> >)':
/tmp/tmp95kpdu0k.cpp:11:5: error: '____' was not declared in this scope
11 |
```

```
   |      ____;
   |      ^~~~
```

```
[C++ kernel] Error: Unable to compile the source code. Return error: 0  
x1.
```

Exercise 3: Use `put()` and in-order traversal to sort a vector in $O(n \log n)$ average time. State why the worst case is $O(n^2)$ for an unbalanced BST.

In [11]:

```
#include <iostream>
#include <vector>
#include "pythonds3/cppds/bst.hpp"
using namespace std;

vector<string> treeSort(vector<string> values) {
    BinarySearchTree bst;
    vector<string> result;
    ____; // 1. insert all elements (O(log n) each on average)
    ____; // 2. in-order traversal visits keys in sorted order (O(n))
    return result;
}

int main() {
    for (string s : treeSort({"t", "a", "o", "j", "d", "b", "q", "f", "l"}))
        cout << s << " ";
    return 0;
}
```

```
/tmp/tmp95kpdu0k.cpp: In function 'std::vector<std::__cxx11::basic_string<char> > treeSort(std::vector<std::__cxx11::basic_string<char> >)':
/tmp/tmp95kpdu0k.cpp:11:5: error: '____' was not declared in this scope
11 |
```

```
    |
    | ____;
    | ^~~~
```

```
[C++ kernel] Error: Unable to compile the source code. Return error: 0x1.
```

Expected output: `a b d f j l o q t` — the in-order walk of a BST is always sorted.

8.14. Search Tree Analysis

Let's first look at the `put()` method. The limiting factor on its performance is the height of the binary tree. Recall from the vocabulary section that the height of a tree is the number of edges between the root and the deepest leaf node.

Let's first look at the `put()` method. The limiting factor on its performance is the height of the binary tree. Recall from the vocabulary section that the height of a tree is the number of edges between the root and the deepest leaf node.

The height is the limiting factor because when we are searching for the appropriate place to insert a node into the tree, we will need to do at most one comparison at each level of the tree!

Let's first look at the `put()` method. The limiting factor on its performance is the height of the binary tree. Recall from the vocabulary section that the height of a tree is the number of edges between the root and the deepest leaf node.

The height is the limiting factor because when we are searching for the appropriate place to insert a node into the tree, we will need to do at most one comparison at each level of the tree!

Note that if the keys are added in a random order, the height of the tree is going to be around $\log_2 n$ where n is the number of nodes in the tree. This is because if the keys are randomly distributed, about half of them will be less than the root and about half will be greater than the root.

Remember that in a binary tree there is one node at the root, two nodes in the next level, and four at the next. The number of nodes at any particular level is 2^d where d is the depth of the level. The total number of nodes in a perfectly balanced binary tree is $2^{h+1} - 1$, where h represents the height of the tree.

Remember that in a binary tree there is one node at the root, two nodes in the next level, and four at the next. The number of nodes at any particular level is 2^d where d is the depth of the level. The total number of nodes in a perfectly balanced binary tree is $2^{h+1} - 1$, where h represents the height of the tree.

A perfectly balanced tree has the same number of nodes in the left subtree as the right subtree. In a balanced binary tree, the worst-case performance of `put()` is $O(\log_2 n)$.

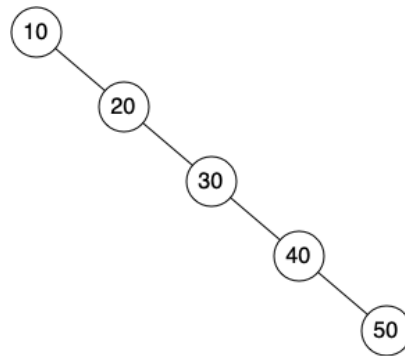
Remember that in a binary tree there is one node at the root, two nodes in the next level, and four at the next. The number of nodes at any particular level is 2^d where d is the depth of the level. The total number of nodes in a perfectly balanced binary tree is $2^{h+1} - 1$, where h represents the height of the tree.

A perfectly balanced tree has the same number of nodes in the left subtree as the right subtree. In a balanced binary tree, the worst-case performance of `put()` is $O(\log_2 n)$.

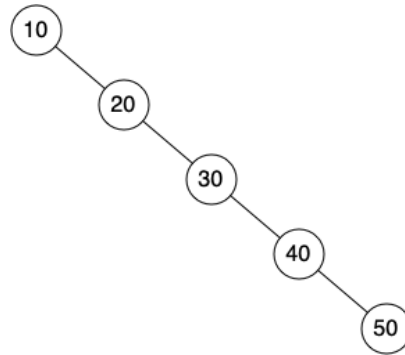
Notice that this is the inverse relationship to the calculation in the previous paragraph. So $\log_2 n$ gives us the height of the tree and represents the maximum number of comparisons that `put()` will need to do as it searches for the proper place to insert a new node.

Unfortunately it is possible to construct a search tree that has height $n - 1$ (for n nodes) simply by inserting the keys in sorted order! An example of this is shown below:

Unfortunately it is possible to construct a search tree that has height $n - 1$ (for n nodes) simply by inserting the keys in sorted order! An example of this is shown below:



Unfortunately it is possible to construct a search tree that has height $n - 1$ (for n nodes) simply by inserting the keys in sorted order! An example of this is shown below:



In this case the performance of the put method is $O(n)$.

The height also limits `get()`, `contains()`, and `remove()`. In the worst case, each follows a root-to-leaf path of length h .

The height also limits `get()`, `contains()`, and `remove()`. In the worst case, each follows a root-to-leaf path of length h .

`remove()` may additionally follow a path to the successor, but that is at most another $O(h)$ traversal. Constant multiples do not change the bound: an unbalanced BST is $O(n)$ worst case, while an AVL tree is $O(\log n)$.

```
In [ ]: display_quiz(path+"bst.json", max_width=800)
```

In a Binary Search Tree (BST), what property must hold for every node to ensure efficient searching?

- ☐ Every node must have exactly two children to maintain a balance.
- ☐ All leaf nodes must be at the same depth level.
- ☐ The keys must be inserted in sorted order to create the most efficient tree.
- ☐ The key of a node must be greater than the keys in its left subtree and smaller than the keys in its right subtree.

What should put(key, value) do when key already exists in the course BST map?

- ☐ Replace the existing value and leave size unchanged.
- ☐ Insert a duplicate node in the right subtree and increase size.
- ☐ Throw underflow_error.
- ☐ Delete the entire subtree rooted at that key.

8.15. Balanced Binary Search Trees

In the previous section we looked at building a binary search tree. As we learned, the performance of the binary search tree can degrade to $O(n)$ for operations like `get()` and `put()` when the tree becomes unbalanced.

In the previous section we looked at building a binary search tree. As we learned, the performance of the binary search tree can degrade to $O(n)$ for operations like `get()` and `put()` when the tree becomes unbalanced.

In this section we will look at a special kind of binary search tree that automatically makes sure that the tree remains balanced at all times. This tree is called an AVL tree.

In the previous section we looked at building a binary search tree. As we learned, the performance of the binary search tree can degrade to $O(n)$ for operations like `get()` and `put()` when the tree becomes unbalanced.

In this section we will look at a special kind of binary search tree that automatically makes sure that the tree remains balanced at all times. This tree is called an AVL tree.

An AVL tree implements the `Map` abstract data type just like a regular binary search tree; the only difference is in how the tree performs. To implement our AVL tree we need to keep track of a balance factor for each node in the tree.

We do this by looking at the heights of the left and right subtrees for each node. More formally, we define the balance factor for a node as the difference between the height of the left subtree and the height of the right subtree.

We do this by looking at the heights of the left and right subtrees for each node. More formally, we define the balance factor for a node as the difference between the height of the left subtree and the height of the right subtree.

$$\textit{balance_factor} = \textit{height}(\textit{left_subtree}) - \textit{height}(\textit{right_subtree})$$

We do this by looking at the heights of the left and right subtrees for each node. More formally, we define the balance factor for a node as the difference between the height of the left subtree and the height of the right subtree.

$$\textit{balance_factor} = \textit{height}(\textit{left_subtree}) - \textit{height}(\textit{right_subtree})$$

Using the definition for balance factor given above, we say that a subtree is left-heavy if the balance factor is greater than zero. If the balance factor is less than zero, then the subtree is right-heavy. If the balance factor is zero, then the tree is perfectly in balance.

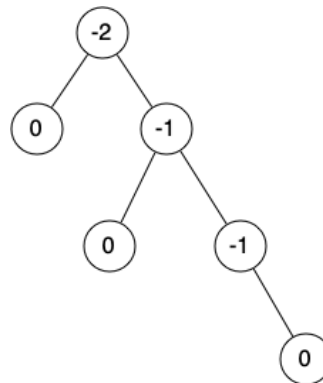
For purposes of implementing an AVL tree and gaining the benefit of having a balanced tree, we will define a tree to be in balance if the balance factor is -1, 0, or 1. Once the balance factor of a node in a tree is outside this range we will need to have a procedure to bring the tree back into balance.

For purposes of implementing an AVL tree and gaining the benefit of having a balanced tree, we will define a tree to be in balance if the balance factor is -1, 0, or 1. Once the balance factor of a node in a tree is outside this range we will need to have a procedure to bring the tree back into balance.

Below shows an example of an unbalanced right-heavy tree and the balance factors of each node:

For purposes of implementing an AVL tree and gaining the benefit of having a balanced tree, we will define a tree to be in balance if the balance factor is -1, 0, or 1. Once the balance factor of a node in a tree is outside this range we will need to have a procedure to bring the tree back into balance.

Below shows an example of an unbalanced right-heavy tree and the balance factors of each node:



8.16. AVL Tree Performance (Optional)

***Course status: optional.** Requiring every balance factor to be -1, 0, or 1 bounds AVL-tree height by $O(\log n)$ (a convenient concrete bound is $h < 1.44 \log_2(n + 1)$). Search, insertion, and the upward balance-update path are therefore logarithmic.*

8.17. AVL Tree Implementation (Optional)

Course status: optional. After inserting a new leaf, update balance factors toward the root. LL and RR imbalances need one rotation; LR and RL imbalances need two. Each local rotation is $O(1)$, while finding and updating the affected path is $O(\log n)$. The course design connects this subclass through the protected virtual `insertOrAssign()` hook.

8.18. Summary of Map ADT Implementations

This chapter connected hierarchical examples to recursive traversals, expression trees, binary-heap priority queues, BST maps, and optional AVL balancing. The table compares a sorted vector, hash table, BST, and AVL tree for the Map ADT operations.

This chapter connected hierarchical examples to recursive traversals, expression trees, binary-heap priority queues, BST maps, and optional AVL balancing. The table compares a sorted vector, hash table, BST, and AVL tree for the Map ADT operations.

operation	Sorted Vector	Hash Table (average)	Binary Search Tree (worst)	AVL Tree
<code>put()</code>	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$
<code>get()</code>	$O(\log n)$	$O(1)$	$O(n)$	$O(\log n)$
<code>contains()</code>	$O(\log n)$	$O(1)$	$O(n)$	$O(\log n)$
<code>remove()</code>	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$

Deletion from a sorted vector is $O(n)$ because later elements must shift.

```
In [ ]: from jupytercards import display_flashcards  
display_flashcards("flashcards/ch9.json")
```

Trees



Next

>

References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds), Chapter 8 (Trees and Tree Algorithms) —
<https://runestone.academy/ns/books/published/cppds/index.html>

